Cheatsheet di Ingegneria del Software A.A. 2022/2023

Indice

- Java
- JML
- Principio di sostituzione
- Programmazione concorrente
- Programmazione funzionale
- Design pattern
- Testing
- UML

Java

Le classi

Vediamo un esempio di classe che rappresenta una data.

```
public class Data {
  private int giorno;
  private int mese;
  private int anno;

  // Restituisce il giorno
  public int ottieniGiorno() {
    return giorno;
  }

  // Restituisce il mese
  public int ottieniMese() {
    return mese;
  }

  // Restituisce l'anno
  public int ottieniAnno() {
    return anno;
  }
}
```

Una classe può essere interpretata come un tipo definito dall'utente che specifica anche le operazioni che vi si possono effettuare. Questo tipo può essere usato per dichiarare altre variabili, ad esempio: `Date d;`.

Gli **attributi** della classe `Data` sono: `giorno`, `mese` e `anno`. I **metodi** invece sono: `ottieniGiorno`, `ottieniMese`, `ottieniAnno`.

In Java è possibile invocare i metodi tramite la **dot notation**, ad esempio:

```
Data d = new Data();
int x;

x = d.ottieniGiorno();
```

Invocare un metodo su un oggetto può cambiarne lo stato. Supponiamo che la classe `Data` si trovi in questo stato iniziale:

```
System.out.println(d.ottieniGiorno() + "/" +
d.ottieniMese() + "/" d.ottieniAnno());
```

```
30/11/2011
```

e che siano stati definiti due nuovi metodi:

```
public Class Data {
  ...
  private int numeroGiorni() {
    switch (mese) {
    case 1:
      return 31;
    case 2:
      return 28;
    ...
    case 12:
      return 31;
    default:
      return -1;
    }
  }
  ...
  private void giornoDopo {
    giorno++;
    if (giorno > numeroGiorni()) {
      giorno = 1;
      mese++;
    }
    if (mese > 12) {
      mese = 1;
      anno++;
    }
  }
  ...
}
```

Allora se invochiamo `giornoDopo`:

```
d.giornoDopo();

System.out.println(d.ottieniGiorno() + "/" +
d.ottieniMese() + "/" d.ottieniAnno());
```

```
1/12/2011
```

Ereditarietà

L'**ereditarietà** è il meccanismo tramite il quale una classe può *estendere* un'altra classe, ereditando tutte le sue variabili e metodi (a meno che questi non siano stati dichiarati come `private`). Ciò significa che una classe figlia può usare tutto ciò che è stato definito nella classe padre, e può anche aggiungere nuove funzionalità. Per farlo si utilizza la keyword `extends`.

È inoltre possibile sovrascrivere i metodi della classe padre, cioè fornire una nuova implementazione per un metodo che è già stato definito nella superclasse. Per farlo, si può utilizzare l'annotazione `@Override` sopra al metodo che si vuole sovrascrivere.

Ad esempio consideriamo le seguenti classi:

```
public class Animal {
  private String name;
  private int age;

  public Animal(String name, int age) {
    this.name = name;
    this.age = age;
  }

  public void makeNoise() {
    System.out.println("Some noise");
  }
}

public class Dog extends Animal {
  private boolean canBark;

  public Dog
  (String name, int age, boolean canBark) {
    super(name, age);
    this.canBark = canBark;
  }

  @Override
  public void makeNoise() {
    if (canBark) {
      System.out.println("Bark bark!");
    } else {
      System.out.println("No barking");
    }
  }
}
```

In questo esempio, la classe `Dog` eredita da `Animal`, il che significa che può usare le variabili `name` e `age` e il metodo `makeNoise()`. Inoltre, `Dog` ha una propria variabile `canBark` e una propria implementazione del metodo `makeNoise()`, che sovrascrive quella presente nella classe padre.

```
Animal a = new Animal("Fluffy", 5);
a.makeNoise(); // stampa "Some noise"

Dog d = new Dog("Max", 3, true);
d.makeNoise(); // stampa "Bark bark!"
```

Binding dinamico e Method resolution

In Java (riguardo all'esempio di prima), potrei anche creare una variabile di tipo `Animal` e assegnarle un'istanza di `Dog`:

```
Animal a = new Dog("Max", 3, true);
a.makeNoise(); // stampa "Bark bark!"
```

ammissibile purché `Dog` sia una sottoclasse di `Animal` (e non il contrario, ad esempio `Dog a = new Animal("Max", 3)` risulta **errato**). In questo caso, `Animal` è il **tipo statico** della variabile `a`, mentre `Dog` è il **tipo dinamico**.
Quando viene invocato il metodo `makeNoise()`, il compilatore non sa quale metodo invocare, poiché non sa quale sia il tipo dinamico della variabile `a` (che viene determinato a runtime). Per questo motivo, Java utilizza il **binding dinamico**.

I passi che segue Java per decidere quale metodo invocare sono i seguenti:

1. Il compilatore controlla se il tipo **statico** della variabile contiene il metodo richiesto (con la stessa signature). Se non è presente, si ha un errore di compilazione. (in questo esempio, Animal ha il metodo `makeNoise()`, quindi si procede al punto 2.).
2. A runtime, Java invece controlla il tipo **dinamico** della variabile. Se questo ha una sua implementazione del metodo richiesto, quella viene utilizzata; altrimenti, si sale lungo la gerarchia delle classi, partendo dal tipo **dinamico**, fino a trovare una classe che ha una implementazione del metodo richiesto (con la stessa signature), o fino a raggiungere la classe del tipo **statico** (che sappiamo contenere per forza il metodo). (in questo esempio, la classe del tipo dinamico `Dog` ha già una sua implementazione di `makeNoise()`, quindi viene scelta quella).
3. Viene invocato il primo metodo trovato.

`public`, `private`, `protected`, friendly

In Java, ci sono diverse parole chiave che possono essere utilizzate per modificare la visibilità di una classe, di un attributo o di un metodo.

- **`public`**: quando una classe, un attributo o un metodo è dichiarato come `public`, significa che esso è accessibile da qualsiasi altra classe o codice all'interno del programma.
- **`private`**: quando una classe, un attributo o un metodo è dichiarato come `private`, significa che esso è accessibile solo all'interno della classe in cui è dichiarato.
- **`protected`**: quando una classe, un attributo o un metodo è dichiarato come `protected`, significa che esso è accessibile nello stesso package, ma solo all'interno della classe in cui è dichiarato e dalle sottoclassi di quella classe.
- **friendly** (o **`default`**): si dichiara non scrivendo nient'altro, significa che esso è accessibile solo all'interno del package in cui è dichiarato.

`static` e `final`

- **`static`**: un attributo o un metodo dichiarato come `static` appartiene alla classe e non agli oggetti di quella classe. Ciò significa che non è necessario creare un'istanza della classe per accedere a un attributo statico o chiamare un metodo statico. Ad esempio, si può avere una classe di utilità per la conversione delle temperature, con un attributo statico per la costante del gas perfetto:

```
public class ConversioneTemperature {
  public static final double
    COSTANTE_GAS_PERFETTO = 8.3145;

  public static double kelvinToCelsius
  (double temperaturaInKelvin) {
      return temperaturaInKelvin - 273.15;
  }
}

/* Si può quindi accedere all'attributo
COSTANTE_GAS_PERFETTO e chiamare il metodo
kelvinToCelsius() senza creare un'istanza
della classe ConversioneTemperature */
```

- **`final`**: quando una classe è dichiarata come `final`, significa che non può essere estesa (cioè, non può avere sottoclassi). Quando un attributo è dichiarato come `final`, significa che ha un valore costante e non può essere modificato una volta assegnato. Quando un metodo è dichiarato come `final`, significa che non può essere sovrascritto da una sottoclasse.

`abstract` e `interface`

- **`abstract`**: Una classe dichiarata come `abstract` non può essere istanziata (cioè, non puoi creare oggetti da essa). Invece, serve come base per le sottoclassi. Una classe astratta può avere metodi sia astratti che concreti (cioè, con corpo). I metodi astratti devono essere implementati dalle sottoclassi, mentre i metodi concreti possono essere utilizzati direttamente dalle sottoclassi senza alcuna implementazione aggiuntiva.

- **`interface`**: Un'**interfaccia** è una classe speciale che contiene solo metodi astratti (cioè, senza corpo). Una classe può implementare un'interfaccia (scrivendo `implements`, invece di `extends`) ; per farlo **deve** definirne tutti i metodi. Le interfacce sono spesso utilizzate per definire un contratto a cui le classi devono aderire.

`this` e `super`

- La parola chiave **`this`** fornisce un riferimento all'oggetto corrente su cui è stato invocato il metodo che stiamo implementando. Può risultare utile quando tale metodo acquisisce dei parametri in input che hanno lo stesso nome di alcuni attributi della classe a cui appartiene: attraverso la keyword `this` possiamo accedere a questi attributi della classe che altrimenti risulterebbero "oscurati" dai parametri. Ad esempio:

```
public class Persona {
  private String nome;
  private int età;

  public Persona(String nome, int età) {
    /* Assegna i valori di nome e età
    agli attributi dell'oggetto corrente */
    this.nome = nome;
    this.età = età;
  }

  ...
}
```

Nell'esempio sopra, `this` viene utilizzato nel costruttore per fare riferimento agli attributi dell'oggetto corrente.

- La parola chiave **`super`** si riferisce alla **superclasse** di una sottoclasse. Può essere utilizzata per fare riferimento a metodi e attributi della superclasse all'interno della sottoclasse. Una delle principali ragioni per cui si utilizza `super` è per chiamare il costruttore della superclasse nel costruttore della sottoclasse. Ciò viene fatto utilizzando la sintassi `super(arg1, arg2, ...)`, dove gli argomenti sono i parametri del costruttore della superclasse.

Casting ed errori a runtime (`ClassCastException`)

Il **casting** in Java è un'operazione che consiste nel convertire un riferimento di un tipo di dato in un riferimento di un altro tipo di dato. Esistono due tipi di casting:

- **Upcasting**: è la conversione di un riferimento a un tipo di dato più specifico in un riferimento a un tipo di dato più generico. Ad esempio:

```
String s = new String("Ciao");
Object o = s;
```

In questo esempio, si sta convertendo il riferimento a un oggetto `String` in un riferimento a un oggetto `Object`. L'upcasting viene eseguito in modo implicito e non richiede l'utilizzo delle parentesi tonde.
Attenzione: È lecito solo se il tipo di dato al quale si sta convertendo il riferimento è un supertipo del tipo di dato del riferimento originale (per le regole dell'ereditarietà).

- **Downcasting**: è la conversione di un riferimento a un tipo di dato più generico in un riferimento a un tipo di dato più specifico. Ad esempio:

```
Object o = new String("Ciao");
String s = (String) o;
```

In questo esempio, si sta convertendo il riferimento a un oggetto `Object` in un riferimento a un oggetto `String`. Il downcasting viene eseguito in modo esplicito e richiede l'utilizzo delle parentesi tonde.
Attenzione: Può lanciare un'eccezione (errore a runtime) di tipo `ClassCastException` se eseguito impropriamente.

`ClassCastException`

I cast eseguiti impropriamente possono generare errori a runtime (eccezioni) di tipo `ClassCastException`. Si verificano quando si tenta di convertire un riferimento di un oggetto in un tipo di riferimento che non è compatibile con l'oggetto a cui fa riferimento.
Ad esempio, questo può accadere quando si tenta di fare un cast da un oggetto di una classe a un oggetto di una classe figlia, ma l'oggetto non è effettivamente un'istanza della classe figlia. In questo caso, verrà generata un'eccezione di tipo `ClassCastException` a runtime. Ad esempio, in questo codice:

```
Object o = new Object();
String s = (String) o;
```

l'oggetto `o` non è un'istanza della classe `String`, quindi il cast non è valido e verrà generata un'eccezione di tipo `ClassCastException` a runtime. Per evitare questo tipo di eccezione, è necessario assicurarsi che l'oggetto a cui fa riferimento il riferimento originale sia effettivamente un'istanza della classe a cui si sta tentando di fare il cast. Ad esempio:

```
Object o = new String("Ciao");
String s = (String) o;
// ok, o è un'istanza di String
```

Collections

Le **Collections** permettono di gestire insiemi di oggetti.

`List<E>`

La classe **`java.util.List`** è un'interfaccia che rappresenta una lista ordinata di elementi. Ecco i metodi disponibili per questa interfaccia:

- **`void add(int index, E element)`**: aggiunge un elemento alla posizione specificata nella lista. Restituisce nulla.
- **`boolean add(E e)`**: aggiunge un elemento alla fine della lista. Restituisce true se l'elemento viene aggiunto con successo, false altrimenti.
- **`boolean addAll(Collection<? extends E> c)`**: aggiunge tutti gli elementi di una collezione alla fine della lista. Restituisce true se almeno un elemento viene aggiunto con successo, false altrimenti.
- **`boolean addAll(int index, Collection<? extends E> c)`**: aggiunge tutti gli elementi di una collezione alla posizione specificata nella lista. Restituisce true se almeno un elemento viene aggiunto con successo, false altrimenti.
- **`void clear()`**: rimuove tutti gli elementi dalla lista. Restituisce nulla.
- **`boolean contains(Object o)`**: verifica se l'oggetto specificato è presente nella lista. Restituisce true se l'oggetto è presente nella lista, false altrimenti.
- **`boolean containsAll(Collection<?> c)`**: verifica se tutti gli elementi di una collezione sono presenti nella lista. Restituisce true se tutti gli elementi sono presenti nella lista, false altrimenti.
- **`boolean equals(Object o)`**: verifica se la lista è uguale a un altro oggetto. Restituisce true se l'oggetto è uguale alla lista, false altrimenti. *Attenzione*: se l'oggetto passato è una lista, allora controlla se le liste hanno stesso numero di elementi e se gli elementi sono uguali (le liste sono uguali).
- **`E get(int index)`**: restituisce l'elemento presente nella posizione specificata nella lista.
- **`int indexOf(Object o)`**: restituisce l'indice della prima occorrenza dell'oggetto specificato nella lista. Restituisce -1 se l'oggetto non è presente nella lista.
- **`boolean isEmpty()`**: verifica se la lista è vuota. Restituisce true se la lista è vuota, false altrimenti.
- **`Iterator<E> iterator()`**: restituisce un iteratore per la lista.
- **`int lastIndexOf(Object o)`**: restituisce l'indice dell'ultima occorrenza dell'oggetto specificato nella lista. Restituisce -1 se l'oggetto non è presente nella lista.
- **`ListIterator<E> listIterator()`**: restituisce un iteratore bidirezionale per la lista.
- **`ListIterator<E> listIterator(int index)`**: restituisce un iteratore bidirezionale per la lista a partire dalla posizione specificata.
- **`E remove(int index)`**: rimuove l'elemento presente nella posizione specificata nella lista. Restituisce l'elemento rimosso.
- **`boolean remove(Object o)`**: rimuove la prima occorrenza dell'oggetto specificato nella lista. Restituisce true se l'oggetto viene rimosso con successo, false altrimenti.
- **`boolean removeAll(Collection<?> c)`**: rimuove dalla lista tutti gli elementi presenti in una collezione. Restituisce true se almeno un elemento viene rimosso con successo, false altrimenti.
- **`boolean retainAll(Collection<?> c)`**: rimuove dalla lista tutti gli elementi che non sono presenti in una collezione. Restituisce true se almeno un elemento viene rimosso con successo, false altrimenti.
- **`E set(int index, E element)`**: sostituisce l'elemento presente nella posizione specificata nella lista con un nuovo elemento. Restituisce l'elemento sostituito.
- **`int size()`**: restituisce il numero di elementi presenti nella lista.
- **`List<E> subList(int fromIndex, int toIndex)`**: restituisce una sottolista della lista compresa tra gli indici specificati.
- **`Object[] toArray()`**: restituisce un array di oggetti contenente tutti gli elementi della lista.
- **`<T> T[] toArray(T[] a)`**: restituisce un array di tipo specificato contenente tutti gli elementi della lista.

Le principali implementazioni dell'interfaccia sono `ArrayList` e `LinkedList`.

`Set<E>`

La classe **`java.util.Set`** è un'interfaccia che rappresenta un insieme di elementi, ovvero una raccolta di elementi **senza duplicati**. Ecco i metodi disponibili per questa interfaccia:

- **`boolean add(E e)`**: aggiunge un elemento all'insieme. Restituisce true se l'elemento viene aggiunto con successo, false altrimenti.
- **`boolean addAll(Collection<? extends E> c)`**: aggiunge tutti gli elementi di una collezione all'insieme. Restituisce true se almeno un elemento viene aggiunto con successo, false altrimenti.
- **`void clear()`**: rimuove tutti gli elementi dall'insieme. Restituisce nulla.
- **`boolean contains(Object o)`**: verifica se l'oggetto specificato è presente nell'insieme. Restituisce true se l'oggetto è presente nell'insieme, false altrimenti.
- **`boolean containsAll(Collection<?> c)`**: verifica se tutti gli elementi di una collezione sono presenti nell'insieme. Restituisce true se tutti gli elementi sono presenti nell'insieme, false altrimenti.
- **`boolean equals(Object o)`**: verifica se l'insieme è uguale a un altro oggetto. Restituisce true se l'oggetto è uguale all'insieme, false altrimenti. *Attenzione*: anche qui, se l'oggetto passato è un Set, allora viene verificato se l'insieme è equivalente a quello passato per parametro.
- **`int hashCode()`**: restituisce il codice hash per l'insieme.
- **`boolean isEmpty()`**: verifica se l'insieme è vuoto. Restituisce true se l'insieme è vuoto, false altrimenti.
- **`Iterator<E> iterator()`**: restituisce un iteratore per l'insieme.
- **`boolean remove(Object o)`**: rimuove l'oggetto specificato dall'insieme. Restituisce true se l'oggetto viene rimosso con successo, false altrimenti.
- **`boolean removeAll(Collection<?> c)`**: rimuove dall'insieme tutti gli elementi presenti in una collezione. Restituisce true se almeno un elemento viene rimosso con successo, false altrimenti.
- **`boolean retainAll(Collection<?> c)`**: rimuove dall'insieme tutti gli elementi che non sono presenti in una collezione. Restituisce true sse almeno un elemento viene rimosso con successo.
- **`int size()`**: restituisce il numero di elementi presenti nell'insieme.
- **`Object[] toArray()`**: restituisce un array di oggetti contenente tutti gli elementi dell'insieme.

La principale implementazione dell'interfaccia è `HashSet`.

Un metodo importante ereditato da Object e che viene ridefinito allo stesso modo dalle implementazioni precedenti di List e Set è **`String toString()`**: data una Lista/Set di elementi A, B, ..., F restituisce "[A.toString(), B.toString(), ..., F.toString()]".

Iterare su un insieme

Per iterare su un insieme (classe `Set`), si può utilizzare il metodo **`iterator()`** della classe `Set`. Questo metodo restituisce un'istanza dell'interfaccia `Iterator`, che permette di scorrere gli elementi dell'insieme uno alla volta. Ecco un esempio di come utilizzare l'iteratore per stampare gli elementi di un insieme:

```
Set<String> set = new HashSet<>();
// aggiungi elementi al set

// otteniamo un iteratore per l'insieme
Iterator<String> iterator = set.iterator();

/* utilizziamo l'iteratore
per scorrere gli elementi dell'insieme */
while (iterator.hasNext()) {
  /* il metodo next() restituisce l'elemento
  corrente e sposta l'iteratore
  all'elemento successivo */
  String element = iterator.next();
  System.out.println(element);
}
```

In questo esempio, viene creato un insieme di stringhe e viene inizializzato un iteratore per l'insieme utilizzando il metodo `iterator()`. Quindi, all'interno del ciclo while, il metodo `hasNext()` viene utilizzato per verificare se ci sono ancora elementi da scorrere nell'insieme. Se ci sono ancora elementi, il metodo `next()` restituisce l'elemento corrente e sposta l'iteratore all'elemento successivo, quindi viene stampato a schermo.

È importante notare che il metodo `next()` può lanciare un'eccezione `NoSuchElementException` se l'iteratore è alla fine dell'insieme e non ci sono più elementi da scorrere. Per questo motivo, è consigliabile utilizzare il metodo `hasNext()` per verificare se ci sono ancora elementi prima di chiamare `next()`.

Inoltre, l'interfaccia `Iterator` fornisce altri metodi utili per la gestione dell'iterazione, come ad esempio:

- `remove()`: rimuove l'elemento corrente dall'insieme. Questo metodo può essere chiamato solo dopo che `next()` è stato chiamato per restituire l'elemento corrente. Lancerà un'eccezione `IllegalStateException` se viene chiamato prima di `next()` o dopo che `hasNext()` restituisce false.
- `forEachRemaining(Consumer<? super E> action)`: esegue un'azione specificata per ogni elemento rimanente nell'iteratore. Questo metodo è utile quando si vuole eseguire un'azione per ogni elemento dell'insieme senza dover gestire manualmente l'iterazione.

Ecco un esempio di come utilizzare questi metodi per rimuovere gli elementi pari dall'insieme:

```
Set<Integer> set = new HashSet<>(Arrays
  .asList(1, 2, 3, 4, 5));

Iterator<Integer> iterator = set.iterator();
while (iterator.hasNext()) {
  int element = iterator.next();
  if (element % 2 == 0) {
    iterator.remove();
  }
}

/* il set ora contiene solo elementi dispari:
[1, 3, 5] */
```

`Map<K,V>`

La classe **`java.util.Map`** è un'interfaccia che rappresenta una mappa di chiavi (di tipo K) a valori (di tipo V). Ecco i metodi disponibili per questa interfaccia:

- **`void clear()`**: rimuove tutte le coppie chiave-valore dalla mappa. Restituisce nulla.
- **`boolean containsKey(Object key)`**: verifica se la chiave specificata è presente nella mappa. Restituisce true se la chiave è presente nella mappa, false altrimenti.
- **`boolean containsValue(Object value)`**: verifica se il valore specificato è presente nella mappa. Restituisce true se il valore è presente nella mappa, false altrimenti.
- **`Set<Map.Entry<K,V>> entrySet()`**: restituisce un insieme di tutte le coppie chiave-valore presenti nella mappa.
- **`V get(Object key)`**: restituisce il valore associato alla chiave specificata nella mappa. Restituisce null se la chiave non è presente nella mappa.
- **`boolean isEmpty()`**: verifica se la mappa è vuota. Restituisce true se la mappa è vuota, false altrimenti.
- **`Set<K> keySet()`**: restituisce un insieme di tutte le chiavi presenti nella mappa.
- **`V put(K key, V value)`**: aggiunge una coppia chiave-valore alla mappa o sostituisce il valore associato alla chiave specificata se già presente. Restituisce il valore precedentemente associato alla chiave, oppure null se la chiave non era presente nella mappa.
- **`void putAll(Map<? extends K,? extends V> m)`**: aggiunge tutte le coppie chiave-valore di un'altra mappa alla mappa corrente. Restituisce nulla.
- **`V remove(Object key)`**: rimuove la coppia chiave-valore associata alla chiave specificata dalla mappa. Restituisce il valore associato alla chiave, oppure null se la chiave non era presente nella mappa.
- **`int size()`**: restituisce il numero di coppie chiave-valore presenti nella mappa.
- **`Collection<V> values()`**: restituisce una collezione di tutti i valori presenti nella mappa.

Le principali implementazioni di Map sono `HashMap`, `LinkedMap` e `TreeMap`. Si noti come nella prima implementazione non è definito un ordine tra elementi, nella seconda l'ordine è quello di inserimento e nella terza l'ordine ASCII tra chiavi.

Data una Map di elementi (kA,A), (kB,B), ..., (kF,F) il metodo **`String toString()`** delle implementazioni restituisce "{kE.toString():E.toString(), kB.toString():B.toString(), ..., kC.toString():C.toString()}" nell'ordine stabilito.

Arrays

Un array di elementi E viene dichiarato e inizializzato nel seguente modo:

```
//dichiarazione
E[] anArray;
//inizializzazione
anArray = new E[SIZE];
//inizializzazione di un certo elemento
anArray[INDEX] = new E();
//dichiarazione di un array bidimensionale
E[][] a2dimArray;
//inizializzazione di un array bidimensionale
a2dimArray = new E[SIZE1][SIZE2];
//dichiarazione e inizializzazione
//di un array di interi
int[] intArray = {0, 4, 3, -2};
```

Si noti come con la prima istruzione viene allocato solamente lo spazio per un riferimento sullo stack, con la seconda lo spazio per SIZE riferimenti sullo heap e con la terza lo spazio per un oggetto E sullo heap. Nel caso di array di tipi primitivi, nella seconda istruzione viene invece allocato spazio sullo heap per contenere SIZE elementi, la terza istruzione (sostituendo new E() con il valore dell'elemento) non alloca quindi spazio in memoria.

Gli array hanno un attributo pubblico **`int length`** che contiene la lunghezza dell'array.

Il metodo **`String toString()`**, applicato su un array di elementi A, B, ..., F restituisce "{A.toString(), B.toString(), ..., F.toString()}".

JML

JML (Java Modeling Language) è un linguaggio di specifica formale.

Sintassi

Lo schema generale di una specifica in JML è il seguente:

```
// OVERVIEW: descrizione della specifica
//@ assignable <parametri che possono essere
//@    modificati dal metodo>;
//@ requires <precondizione>
//@ ...;
//@ ensures <postcondizione>
//@ ...;
//@ signals (<tipo eccezione 1> e)
//@ <postcondizione nel caso di eccezione 1>
//@ ...;
//@ signals (<tipo eccezione 2> e)
//@ <postcondizione nel caso di eccezione 2>
//@ ...;
```

Ogni riga della specifica (dopo l'`OVERVIEW`) inizia con `//@` . Alla fine di ciascun "blocco" (`assignable`, `requires`, ...) è necessario aggiungere `;`. Omettere il blocco `requires` equivale a scrivere `//@ requires true;`. Omettere il blocco `ensures` equivale a scrivere `//@ ensures true;`. Nella condizione del blocco `signals` possiamo fare riferimento all'oggetto eccezione `e` (il nome è arbitrario).

Condizioni

Le condizioni in JML (precondizioni e postcondizioni) sono espressioni booleane di Java (con alcuni operatori e quantificatori aggiuntivi) che non alterano lo stato degli oggetti che descrivono: non si possono usare assegnamenti (=), incrementare le variabili intere (++), etc. Gli unici metodi che si possono invocare sono quelli che non modificano lo stato degli oggetti a cui appartengono o dei parametri che gli vengono passati (si dicono **metodi puri**).

Operatori booleani aggiuntivi

Siano `a` e `b` due condizioni JML, allora le seguenti sono valide condizioni JML:

- `a ==>` `b`: a implica `b`
- `a <==` `b`: `b` implica `a`
- `a <==>` `b`: a se e solo se `b`
- `a <=!=>` `b`: `!(a <==> b)`

`\old`

L'operatore `\old(<espressione>)` prende in input una espressione che può essere una condizione JML oppure una espressione Java che non ha "effetti collaterali" (non si può utilizzare ++, =, metodi **non** puri, etc.) e restituisce il risultato che si ottiene valutando tale espressione nel momento della chiamata del metodo che stiamo specificando. Notare che, valutando una espressione Java (tra quelle ammissibili), `\old` in generale non restituisce valori booleani. Non ha senso utilizzare `\old` nel blocco `requires` dato che lì stiamo specificando ciò che deve essere vero **prima** che il metodo venga eseguito.

Quantificatori

Sia `T` un generico tipo di Java ed `a(t)`, `b(t)` due condizioni JML definite su un oggetto `t` di tipo `T` (ad esempio se `T = int`, `a(t) = t > 1729`), allora le seguenti sono valide condizioni JML:

- `(\forall T t; a(t); b(t))`: per ogni oggetto del tipo `T` se `a(t)` allora `b(t)`
- `(\exists T t; a(t); b(t))`: esiste un oggetto del tipo `T` tale che `a(t)` e `b(t)`
- `(\num_of T t; a(t); b(t))`: restituisce il numero di oggetti `t` tale che `a(t)` e `b(t)`

Nel caso `T` non sia un tipo primitivo assumiamo `t != null`.

Operatori di aggregazione

Sia `T` un generico tipo di Java ed `N` uno dei tipi numerici primitivi di Java (`int`, `float`, ...). Sia `a(t)` una condizione JML definita sull'oggetto `t` di tipo `T`. Sia `b : T -> N` una funzione (in senso matematico) implementata attraverso Java. Ad esempio: `T = String`, `N = int`, `a(t) = !t.isEmpty() && t.charAt(0) == 'e'`, `b(t) = t.length()`. Allora possiamo applicare i seguenti operatori di aggregazione:

- `(\sum T t; a(t); b(t))`: restituisce la somma `b(t_1) + b(t_2) + ...` per tutti i `t_i` tali che `a(t_i)`
- `(\product T t; a(t); b(t))`: restituisce il prodotto `b(t_1)*b(t_2)*...` per tutti i `t_i` tali che `a(t_i)`
- `(\max T t; a(t); b(t))`: restituisce il massimo tra `b(t_1), b(t_2), ...` per tutti i `t_i` tali che `a(t_i)`
- `(\min T t; a(t); b(t))`: restituisce il minimo tra `b(t_1), b(t_2), ...` per tutti i `t_i` tali che `a(t_i)`

`\result`

Se stiamo scrivendo la specifica di un metodo NON `void`, possiamo fare riferimento al valore restituito con la keyword `\result`. Ha senso utilizzare `\result` solo nell'`ensures` (in `requires` il metodo non è ancora stato eseguito, `signals` specifica la postcondizione qualora venga restituita un'eccezione).

`assignable`

Il blocco `assignable` nella specifica permette di esplicitare i parametri (di tipo riferimento) che vengono modificati dal metodo. Omettere `assignable` significa che tutti i parametri possono essere modificati.

Se un metodo non modifica nessuno dei parametri possiamo usare la keyword `\nothing`: `//@ assignable \nothing;`.

Consideriamo il seguente metodo:

```
public static void metodo(S s, T t, U u) {
    ...
}
```

Per specificare che `s` e `t` vengono modificati, ma non `u`, possiamo aggiungere nella specifica: `//@ assignable s, t;`

Supponiamo di voler specificare che un array `a[]` venga modificato da un metodo di cui stiamo scrivendo la specifica:

- `//@ assignable a[*];`: ogni elemento di `a` può essere modificato
- `//@ assignable a[5];`: solo l'elemento `a[5]` può essere modificato
- `//@ assignable a[1..8];`: gli elementi `a[1]`, `a[2]`, ..., `a[8]` possono essere modificati

Commenti

A volte è utile inserire all'interno delle condizioni JML delle specifiche informali. Per farlo si utilizzano i commenti, esprimibili attraverso la seguente sintassi: `(* <commento> *)`. Ciascun commento al momento della valutazione della condizione è da intendersi con valore `true`.

Semantica

Precondizione

La **precondizione** è la condizione sulla base della quale è definita la specifica: se la precondizione non dovesse essere rispettata allora non è detto che il metodo si comporti secondo la specifica (in generale non lo farà, qualsiasi comportamento è ammesso). In JML è possibile esprimere una precondizione del metodo che stiamo specificando attraverso il blocco `requires`. Nel caso in cui la precondizione sia `true` il metodo "non ha una precondizione": la sua specifica è valida a prescindere dei parametri passati. Un metodo con precondizione `true` si dice **totale**, altrimenti si dice **parziale**.

Postcondizione

La **postcondizione** esplicita una serie di "effetti" garantiti al termine dell'esecuzione del metodo che stiamo specificando, assumendo che la precondizione fosse verificata. In JML è possibile definire separatamente la postcondizione che vale nel caso in cui il metodo esegua regolarmente dalle postcondizioni che valgono nel caso in cui durante l'esecuzione del metodo si verifichino delle eccezioni. La prima si esplicita attraverso il blocco `ensures`. La seconda attraverso `signals (<tipo eccezione> e)`. Nel caso in cui la postcondizione sia `true` il metodo non garantisce nessun "effetto" al termine della sua esecuzione, quindi può "fare ciò che vuole": non c'è motivo di omettere la postcondizione.

Astrazione procedurale

Si dice **astrazione procedurale** la specifica di un'operazione (anche complessa) definita su un dominio di dati generici (parametri). In Java coincide con la specifica (ad esempio attraverso la notazione JML) di un metodo statico. Infatti un metodo statico rappresenta proprio un'operazione "priva di stato": il suo comportamento non dipende dai valori di attributi non statici, ma è determinato unicamente dai parametri.

ADT: Abstract Data Type

Si dice **abstract data type** (o **ADT**) un tipo di dato per cui tutti gli stati ammissibili e le operazioni che vi si possono effettuare (includendo il modo in cui quest'ultime permettono di passare da uno stato all'altro) sono descritti attraverso una specifica. La specifica di un ADT si compone di una sintassi e di una semantica. La sintassi coincide con l'interfaccia del tipo di dato in questione (l'insieme di metodi pubblici che espone). Per definire la semantica in maniera esaustiva occorre utilizzare un linguaggio di specifica formale come JML. A differenza dell'astrazione procedurale, nello specificare un ADT con JML, dobbiamo trattare anche le transizioni di stato dell'oggetto dovute alle varie operazioni (cioè dire ciò che cambia, come cambia e cosa invece rimane invariato tra i valori che rappresentano lo stato dell'oggetto a seguito dell'esecuzione di un certo metodo).

Metodi puri

Un metodo **di un ADT** si dice **puro** se:

- vale `//@ assignable \nothing;`
- gli unici metodi che invoca sono anche essi puri
- non modifica nessun attributo (pubblico o privato) dell'ADT

I metodi puri si indicano in JML con la keyword `/* @ pure @ */`. Ad esempio, supponiamo che il metodo `dimensione` restituisca la cardinalità di un ADT che rappresenta una collezione di oggetti. `dimensione` è chiaramente un metodo puro, in quanto non modifica lo stato della collezione, per esplicitarlo in JML possiamo usare la sintassi: `public int /* @ pure @ */ dimensione();`.

Anche un costruttore può essere **puro** se si limita ad inizializzare gli attributi dell'oggetto senza apportare altre modifiche.

I metodi puri si dicono anche **observer** in quanto permettono di "osservare" lo stato di un ADT in un determinato momento.

Classi pure

Una classe i cui metodi **pubblici** (e costruttori) sono tutti **puri** si dice **pura** e si indica con la sintassi: `public /* @ pure @ */ class NomeClasse`. Una classe **pura** è una classe **immutabile**.

Attenzione: quando specifichiamo che una classe è **pure NON** è necessario che nella specifica si espliciti che il suo stato non cambia (cioè che tutti gli observer restituiscono gli stessi risultati prima e dopo l'esecuzione del metodo). Questo risulterà utile in seguito per alleggerire la specifica in JML dei metodi.

Specifica di un'operazione di un ADT in JML

Valgono le regole sintattiche e semantiche già introdotte per JML. La struttura della specifica rimane quella usuale.

Attenzione: quando specifichiamo un'operazione di un ADT in JML, nella specifica possono comparire solo gli altri metodi (e attributi) **pubblici** e **puri** (dell'ADT), i parametri formali dell'operazione (su cui possiamo invocare i rispettivi metodi **pubblici** e **puri**) e `\result` per fare riferimento al risultato restituito.

Esempio

Vediamo un esempio di ADT.

Vogliamo descrivere attraverso la specifica un *insieme (mutabile) di interi* su cui sono definite le seguenti operazioni:

- *costruzione*: crea ed inizializza l'insieme come un insieme vuoto
- *inserisci(x)*: aggiunge l'elemento x all'insieme
- *rimuovi(x)*: rimuove l'elemento x dall'insieme
- *appartiene(x)*: restituisce *vero* se x appartiene all'insieme, *falso* altrimenti
- *cardinalità*: restituisce la cardinalità dell'insieme
- *scegli*: restituisce uno tra gli elementi dell'insieme

Alcuni valori ammissibili per l'insieme sono: *{1, 10, 35}*, *{7}*, ...

```
public class InsiemeDiInteri {
  // OVERVIEW: Insieme di interi illimitato
  //  e mutabile

  // Observer:

  //@ ensures (* \result equivale a
  //@ "x appartiene all'insieme" *);
  public /* @ pure @ */
    boolean appartiene(int x);

  //@ ensures \result ==
  //@   (\num_of int x; ; appartiene(x))
  public /* @ pure @ */ int cardinalita();

  //@ ensures appartiene(\result) &&
  //@ cardinalita() > 0;
  //@ signals (EccezioneInsiemeVuoto e)
  //@ cardinalita() == 0;
  public /* @ pure @ */ int scegli()
    throws EccezioneInsiemeVuoto;

  // Creator:

  //@ ensures cardinalita() == 0;
  public InsiemeDiInteri();

  // Mutator:

  //@ ensures appartiene(x) &&
  //@ cardinalita() == \old(cardinalita() +
  //@   (appartiene(x) ? 0 : 1)) &&
  //@ (\forall int y; \old(appartiene(y));
  //@   appartiene(y));
  public void inserisci(int x);

  //@ ensures !appartiene(x) &&
  //@ cardinalita() == \old(cardinalita() -
  //@   (appartiene(x) ? 1 : 0)) &&
  //@ (\forall int y; appartiene(y);
  //@   \old(appartiene(y)));
  public void rimuovi(int x);
}
```

Osserviamo che **NON** tutti i metodi pubblici dell'oggetto possono essere specificati formalmente (ovvero senza dover ricorrere ai commenti) in JML. Questo in particolare è vero per alcuni degli **observer**, nel caso d'esempio: `appartiene`. Per poter specificare formalmente questi metodi sarà necessario dichiarare un **rappresentante**, cioè un oggetto che realizza lo stato concreto dell'ADT, (spiegato in seguito) e fare riferimento all'implementazione.

Un ADT deve consentire di utilizzare gli oggetti che descrive, senza conoscerne l'implementazione. Vediamo come implementare, conoscendo solo la specifica, un metodo che crea un `InsiemeDiInteri` a partire da un array di interi:

```
public static InsiemeDiInteri daArray(int[] a)
  throws NullPointerException {

  if (a == null) {
    throw new NullPointerException();
  }

  InsiemeDiInteri s = new InsiemeDiInteri();
  // Dalla specifica s è vuoto

  for (int x : a) {
    s.inserisci(x);
  }
  // Dalla specifica s contiene tutti
  // e soli gli interi (x) in a

  return s;
}
```

Categorie di operazioni

Possiamo classificare le operazioni (i metodi pubblici) definite su un ADT nelle seguenti categorie:

- ***Creator***: creano un nuovo oggetto del tipo definito dall'ADT prendendo in input parametri che **NON** sono altri oggetti del tipo definito dall'ADT
- ***Producer***: creano un nuovo oggetto del tipo definito dall'ADT prendendo in input un oggetto del tipo definito dall'ADT
- ***Mutator***: modificano lo stato dell'oggetto
- ***Observer***: restituiscono un risultato di un tipo diverso da quello definito dall'ADT; solitamente non modificano l'oggetto e quindi sono dichiarati come puri

Un tipo *mutabile* "adeguato" dovrebbe disporre di *creator*, *observer* e *mutator*.

Un tipo *immutabile* "adeguato" dovrebbe disporre di *creator*, *observer* e *producer*.

Proprietà astratte (evolutive e invarianti)

Si dicono proprietà **astratte** di un ADT tutte le proprietà di cui esso gode, che sono deducibili dalla specifica dei suoi metodi pubblici. Si distinguono in:

- Proprietà **evolutive**: specificano la relazione tra uno stato osservabile ed il successivo (ovvero specificano come l'oggetto evolve). Un esempio di proprietà evolutiva è l'immutabilità, che può essere espressa attraverso il concetto di stato corrente e successivo come segue: "Per ogni stato corrente dell'oggetto, per ogni operazione, detto stato successivo lo stato in cui si trova l'oggetto dopo aver eseguito l'operazione, lo stato corrente coincide con lo stato successivo"

- Proprietà **invarianti**: si tratta di proprietà che sono valide per qualunque stato ammissibile dell'oggetto. Nell'esempio `InsiemeDiInteri`, una proprietà invariante è: `cardinalita() >= 0`.

Le proprietà **evolutive NON** sono **rappresentabili direttamente in JML**: non esiste una sintassi per dichiararle esplicitamente, ma seguono dalle specifiche dei singoli metodi.

Al contrario, per **rappresentare in JML** una proprietà **invariante**, anche detta **invariante pubblico**, si usa la seguente sintassi:

```
public class ClasseADT {
  //@ public invariant <condizione>;

  ...
}
```

Le proprietà astratte permettono agli utenti dell'ADT di fare assunzioni che ne semplificano l'utilizzo.

Dimostrare la validità di un invariante pubblico

Dichiarare un invariante pubblico non è sufficiente per garantirne la validità. Dobbiamo assicurarci, attraverso la specifica, che l'invariante sia valido per tutti gli stati "iniziali" delle istanze dell'ADT: ovvero gli oggetti ottenuti invocando i creator. Successivamente è necessario verificare che, a partire da un generico stato ammissibile dell'oggetto, applicando una qualsiasi delle operazioni definite e assumendo vere le rispettive precondizioni, la proprietà sia ancora valida. In particolare uno stato si considera "ammissibile" se è raggiungibile applicando un numero finito di volte le operazioni dell'ADT ad uno degli stati iniziali. Non è necessario dimostrare che gli observer soddisfino l'invariante, dato che non modificano lo stato dell'oggetto.

Rappresentante

Per **implementare** un ADT è necessario dichiarare un **rappresentante** (o **`rep`**), ovvero una *struttura dati* in grado di rappresentare, attraverso i valori che assume, lo stato concreto dell'ADT. In Java questo si traduce in un insieme di attributi **privati**. Ricordiamo che l'utente deve poter utilizzare l'ADT conoscendo esclusivamente la sua specifica, quindi non è necessario (anzi vedremo essere una pratica scorretta) esporre il rappresentante dello stato concreto (magari dichiarandolo pubblico).

Nell'esempio `InsiemeDiInteri` un rappresentante molto semplice, ma efficace, è un `ArrayList<Integer>` che contiene gli elementi dell'insieme. Vediamo una possibile implementazione:

```
public class InsiemeDiInteri {
  private ArrayList<Integer> rep;

  public boolean appartiene(int x) {
    return trova(x) != -1;
  }

  public int cardinalita() {
    return rep.size();
  }

  public int scegli() throws
      EccezioneInsiemeVuoto {
    if (rep.size() == 0) {
      throw new EccezioneInsiemeVuoto();
    }
    return rep.get(0);
  }

  public InsiemeDiInteri() {
    rep = new ArrayList<>();
  }

  public void inserisci(int x) {
    if (appartiene(x)) { return; }
    rep.add(x);
  }

  public void rimuovi(int x) {
    int iX = trova(x);
    if (iX == -1) { return; }
    rep.remove(iX);
  }

  // trova è un metodo di ausilio che cerca
  // x in rep e, se lo trova
  // restituisce l'indice corrispondente,
  // altrimenti -1
  private /* @ helper @ */ int trova(int x) {
    for (int i = 0; i < rep.size(); i++) {
      if (rep.get(i) == x) { return i; }
    }
    return -1;
  }
}
```

Funzione di astrazione

Si dice **funzione di astrazione** (o **AF**) una funzione (in senso matematico) che associa ad ogni stato concreto ammissibile, rappresentato dal `rep`, lo stato astratto (dell'ADT) che vi corrisponde. Nell'esempio `InsiemeDiInteri` lo stato concreto è un `ArrayList<Integer>` come ad esempio `[4, 1, 2, 3]`. L'AF associa `[4, 1, 2, 3]` all'insieme `{1, 2, 3, 4}` che è proprio lo stato astratto dell'ADT corrispondente a tale `ArrayList`. Le funzioni di astrazione possono risultare più o meno complicate e **dipendono** dall'implementazione scelta. Solitamente sono **NON** iniettive: più stati concreti corrispondono allo stesso stato astratto. Nell'esempio `[4, 1, 2, 3]` e `[1, 2, 3, 4]` corrispondono entrambi all'insieme `{1, 2, 3, 4}`.

Come dichiarare l'AF in JML

In JML è possibile **dichiarare** (**NON** implementare) la funzione di astrazione attraverso la seguente sintassi:

```
public class ClasseADT {
  ...
  // AF:
  //@ private invariant
  //@ <condizione>;
  ...
}
```

L'AF viene dichiarata all'interno del blocco JML **private invariant**, che ci consente di accedere, oltre che agli attributi e metodi pubblici della classe, **anche a quelli privati**. Nella condizione JML siamo interessati ad esplicitare la **relazione** (logica) che sussiste tra lo stato concreto e lo stato astratto corrispondente dell'ADT. Cioè non vogliamo spiegare cosa la funzione fa (la sua implementazione) ma solo le proprietà che devono valere perchè uno stato concreto ed uno astratto siano corrispondenti.

Nell'esempio `InsiemeDiInteri` con un'`ArrayList` come `rep`, a prescindere da come l'AF venga realizzata, vale che *"un elemento `x` appartiene all'insieme `s` (stato astratto) sse esiste un indice `i` compreso tra 0 e `rep.size()` tale che `rep.get(i) == x` (stato concreto)"*. Lo stato concreto è rappresentato dal `rep`, a cui possiamo accedere dato che è costituito da un insieme di attributi privati (ricordiamo che siamo nel private invariant). Lo stato astratto invece non è realmente memorizzato da nessuna parte, è possibile accedervi solo attraverso gli *observer*. Quindi nella condizione JML compariranno gli attributi **privati** che costituiscono il `rep` e gli *observer* che permettono di osservare lo stato astratto dell'ADT, legati tra loro attraverso delle formule logiche che risultano vere solo quando i valori assunti dagli attributi **privati** corrispondono con lo stato astratto osservato. Dichiariamo l'AF nell'esempio `InsiemeDiInteri`:

```
public class InsiemeDiInteri {
  ...
  // AF:
  //@ private invariant
  //@ (\forall int x; ;
  //@   appartiene(x) <==>
  //@   (\exists int i; 0 <= i &&
  //@     i < rep.size(); rep.get(i) == x));
  ...
}
```

Notiamo che siamo finalmente riusciti a specificare formalmente il metodo `appartiene` che, prima dell'introduzione del `rep`, era semplicemente descritto attraverso un commento in JML.

Come implementare l'AF

Per **implementare** l'AF solitamente si ricorre ad una rappresentazione testuale (tramite una stringa) degli stati astratti, quindi si ridefinisce `toString`. Nell'esempio `InsiemeDiInteri` una possibile implementazione dell'AF (attraverso i paradigmi della programmazione funzionale) è quella che segue:

```
public class InsiemeDiInteri {
  ...
  @Override
  public String toString() {
    return rep.stream().sorted()
      .map(x -> x.toString())
      .collect(Collectors
        .joining(", ", "{", "}"));
  }
}
```

Osserviamo che l'ordinamento della lista di interi prima della stampa fa in modo che gli stati astratti corrispondenti a `[1, 2, 3, 4]` e `[4, 3, 2, 1]`, **che sono uguali**, siano rappresentati da un'unica stringa `"{1, 2, 3, 4}"`.

Invariante di rappresentazione

NON tutti gli stati concreti assunti dal `rep` sono rappresentanti ammissibili di uno stato astratto. Nell'esempio `InsiemeDiInteri` risulta evidente che i seguenti valori per il `rep` **NON** costituiscano degli stati ammissibili:

- `null`
- `[1, null, 3]`

Ci sono però altre assunzioni (implicite) che facciamo nell'implementazione di `InsiemeDiInteri` che rendono inammissibili anche altri stati concreti che potrebbero sembrare validi. Ad esempio, quando rimuoviamo un elemento dall'insieme, rimuoviamo dal `rep` al più un elemento (in particolare rimuoviamo l'elemento che vale `x` con indice minimo, se esiste). Quindi, perché l'implementazione funzioni correttamente, è imperativo che nel `rep` non compaiano duplicati (altrimenti la rimozione non rimuoverebbe tutti gli elementi uguali ad `x`). Dunque, ad esempio, anche `[1, 1, 2]` è uno stato concreto inammissibile.

L'**invariante di rappresentazione** (o **RI**) è una proprietà invariante, che quindi deve essere verificata in **tutti** gli stati osservabili dell'istanza dell'ADT (questo punto in particolare sarà chiarito in *Validità e conservazione dell'RI*), privata, cioè che fa riferimento **esclusivamente** agli attributi privati della classe, in particolare a quelli che costituiscono il `rep`, che deve essere soddisfatta perchè lo stato concreto rappresentato dal `rep` sia ammissibile.

Analogamente si può definire l'RI come la proprietà che permette di discernere tra i valori del `rep` che appartengono al dominio della funzione di astrazione (gli stati concreti ammissibili, per cui esiste uno stato astratto corrispondente) da quelli che non vi appartengono (gli stati concreti inammissibili, per cui non esiste uno stato astratto corrispondente e per cui, di conseguenza, la funzione di astrazione risulta indefinita). Ovviamente l'RI dipende fortemente dalle scelte implementative.

Per dichiarare l'invariante di rappresentazione in JML si usa la seguente sintassi:

```
public class ClasseADT {
  ...
  // RI:
  //@ private invariant
  //@ <condizione-sul-rep>;
  ...
}
```

Nella condizione in JML sul `rep` compariranno appunto **solo gli attributi** (privati) **che costituiscono il `rep`**. La condizione risulterà vera *sse* il `rep` rappresenta uno stato concreto ammissibile.

Dichiariamo l'RI di `InsiemeDiInteri`:

```
public class InsiemeDiInteri {
  ...
  // RI:
  //@ private invariant
  //@ rep != null && !rep.contains(null) &&
  //@ (\forall int i; 0 <= i &&
  //@   i < rep.size() - 1;
  //@   (\forall int j; i < j &&
  //@     j < rep.size();
  //@     !rep.get(i).equals(rep.get(j))));
  ...
}
```

Validità e conservazione dell'RI

Gli *stati osservabili*, cioè quelli per cui si verifica la validità dell'RI, sono gli stati in cui si trova il `rep` quando nessun metodo **pubblico** è **in esecuzione**. Questo perché, durante l'esecuzione di un metodo, l'invariante di rappresentazione potrebbe essere temporaneamente violato per motivi implementativi per poi essere ripristinato prima della fine dell'esecuzione.

Ad esempio, consideriamo un'implementazione **alternativa** inefficiente di `inserisci` in `InsiemeDiInteri`:

```
public class InsiemeDiInteri {
  ...
  public void inserisciIneff(int x) {
    int iX = trova(x);
    rep.add(x);
    if (iX != -1) { rep.remove(x); }
  }
  ...
}
```

`inserisciIneff` controlla se `x` sia già un elemento dell'insieme, in caso affermativo lo inserisce in fondo (con `add`) e rimuove il duplicato (con `remove`), altrimenti effettua solo l'inserimento. Nonostante nel primo scenario, tra la chiamata ad `add` e quella a `remove`, l'RI risulti temporaneamente violato, dato che avremo un elemento `x` duplicato in `rep`; l'implementazione è comunque valida perché ripristina l'invariante prima del termine dell'esecuzione del metodo.

Per verificare che l'RI sia valido e si conservi durante il ciclo di vita dell'oggetto dobbiamo innanzitutto verificare che valga dopo l'esecuzione di tutti i costruttori pubblici (in tutti gli stati concreti iniziali). Poi dobbiamo dimostrare che, supponendo che l'RI sia valido per un certo valore del `rep` (quindi supponendo di trovarci in uno stato concreto ammissibile), per ogni metodo pubblico della classe, al termine dell'esecuzione di tale metodo, l'invariante sia ancora rispettato.

Esposizione del `rep`

Si parla di **esposizione del `rep`** quando si fornisce all'esterno un riferimento al `rep` dell'ADT. Si tratta di una pessima pratica di programmazione perché consente agli utilizzatori dell'ADT di violare l'invariante di rappresentazione, agendo direttamente sul riferimento al `rep` di cui dispongono. Avviene principalmente in due modalità (che appaiono innocue):

- Restituire il `rep` attraverso un metodo pubblico. Consideriamo l'esempio `InsiemeDiInteri` e supponiamo di voler implementare il metodo `comeArrayList` che restituisce una rappresentazione dell'insieme sotto forma di `ArrayList<Integer>`. Un'implementazione che potrebbe risultare naturale è:

```
public class InsiemeDiInteri {
  ...
  public ArrayList<Integer> comeArrayList() {
    return rep;
  }
  ...
}
```

Sbagliato! Stiamo esponendo il `rep`.

- Assegnare un riferimento ricevuto come parametro al `rep`. Supponiamo ora di voler implementare il metodo *statico* `daArrayList` che riceve un `ArrayList<Integer>` e restituisce il corrispondente `InsiemeDiInteri`. Una possibile implementazione è:

```
public class InsiemeDiInteri {
  ...
  public static InsiemeDiInteri
    daArrayList(ArrayList<Integer> lista) {
    InsiemeDiInteri s = new InsiemeDiInteri();
    s.rep = lista;
    return s;
  }
  ...
}
```

Anche in questo caso l'implementazione è **sbagliata!** Non solo stiamo esponendo il `rep`, non stiamo neanche controllando che `lista` soddisfi l'RI.

Per ovviare a questi problemi di solito si ricorre ai *copy constructor*. Ad esempio, per evitare l'esposizione del `rep` in `comeArrayList`:

```
public class InsiemeDiInteri {
  ...
  public ArrayList<Integer> comeArrayList() {
    return new ArrayList<Integer>(rep);
  }
  ...
}
```

Trucchi per l'esame

***Attenzione!**: I seguenti "trucchi" sono stati ricavati analizzando pattern ricorrenti nella risoluzione degli esercizi sul JML. **NON** sono algoritmi per risolvere gli esercizi e **NON** garantiscono di trovare la soluzione corretta.*

Prima di procedere alla specifica di un metodo in particolare, risulta utile analizzare la classe nel suo complesso.

Individuare gli *observer "indipendenti"*

Definiamo (è una definizione inventata dagli autori del cheatsheet) **indipendenti** gli *observer* la cui specifica non può essere espressa in funzione di altri *observer* della classe (o di altre classi). Gli *observer indipendenti* possono essere completamente specificati solo attraverso la funzione di astrazione, facendo riferimento al `rep`. Per poter applicare le *tecniche* che seguono è necessario individuare quali sono gli *observer indipendenti* della classe in analisi: solitamente, nei TdE, si tratta di *observer* che restituiscono collezioni (`List` o `Set` nella maggior parte dei casi).

Dopo aver letto **attentamente** la specifica informale in linguaggio naturale, procediamo alla specifica del metodo in JML:

Ricavare la precondizione (`requires`)

- Solitamente nella precondizione si richiede che tutti i parametri in input di tipo riferimento **NON** siano `null`

- Nel caso di parametri di tipo numerico, se rappresentano quantità positive o non negative può essere necessario specificarlo nella precondizione

- Nel caso in cui la classe rappresenta una collezione di oggetti, uno dei parametri potrebbe essere un oggetto della collezione ed, a volte, è necessario specificare che vi faccia parte

Vediamo un esempio **per quest'ultimo caso**: l'esercizio 1 del TdE del 2022-06-18.

Si consideri la classe Java `HotelDB` per gestire la prenotazione di stanze di hotel.

```
public class HotelDB {
  // Ritorna l'elenco di tutti gli hotel
  // contenuti.
  public /*@ pure @*/ Set<Hotel> getHotels();

  // Ritorna la lista di tutte le stanze che
  // (i) possono ospitare almeno people
  // persone, (ii) si trovano vicino a
  // location, (iii) sono libere il giorno
  // day. La lista e' in ordine crescente di
  // prezzo (dalla stanza meno costosa alla
  // piu' costosa).
  public /*@ pure @*/ List<Room>
    findRooms(int people, Location location,
    Day day);

  // Prenota la stanza room per il giorno day.
  // Solleva una AlreadyBookedException se la
  // stanza non e' disponibile il giorno
  // specificato.
  public void book(Room room, Day day)
    throws AlreadyBookedException;
}

public class Hotel {
  // Ritorna tutte le stanze presenti
  // nell'hotel.
  public /*@ pure @*/ Set<Room> getRooms();

  // Ritorna true sse l'hotel e' vicino a
  // location.
  public /*@ pure @*/ boolean
    isCloseTo(Location location);
}

public class Room {
  // Ritorna true sse la stanza e' disponibile
  // il giorno day.
  public /*@ pure @*/ boolean
    isAvailable(Day day);

  // Ritorna il numero massimo di persone che
  // la stanza puo' ospitare.
  public /*@ pure @*/ int getMaxPeople();

  // Ritorna il prezzo della stanza.
  public /*@ pure @*/ float getPrice();

  // Permette di prenotare la stanza nel
  // giorno precisato. Richiede che la stanza
  // non sia gia' prenotata per lo stesso
  // giorno.
  public void book(Day day);
}
```

Nel punto *b* dell'esercizio viene chiesto di specificare il metodo `book` di `HotelDB`. Il metodo `book` ha tra i suoi parametri `room` ovvero la stanza da prenotare; nella precondizione dobbiamo specificare che la stanza in questione si trovi in uno degli hotel nel nostro database:

```
//@ requires room != null && day != null &&
//@ (\exists Hotel h; getHotels()
//@   .contains(h); h.getRooms()
//@   .contains(room));
//@ ...
public void book(Room room, Day day)
  throws AlreadyBookedException;
```

Attenzione!: a volte il metodo che stiamo specificando restituisce delle eccezioni quando uno dei parametri ha dei valori non ammessi (può succedere per tutti i casi elencati); in tal caso **NON** bisogna specificare nella precondizione che il parametro abbia un valore ammissibile: ce ne occuperemo nei blocchi `ensures` e `signals`.

Ricordarsi sempre di leggere **attentamente** la specifica informale in linguaggio naturale, dove potrebbero essere segnalate altre precondizioni particolari, ad hoc per il metodo in questione.

Classificare il metodo

Per semplificare la stesura della specifica in JML risulta conveniente differenziare il tipo di metodo che dobbiamo specificare (vedi il paragrafo *Categorie di operazioni*) nelle seguenti categorie:

- *Mutator che non restituiscono nulla*
- *Observer puri*
- *Mutator che restituiscono qualcosa*
- *Creator*
- *Producer in classi pure (immutabili)*

Ricavare la postcondizione normale (`ensures`)

Oltre a ciò che è espresso nella specifica informale in linguaggio naturale, nella specifica formale in JML dobbiamo esplicitare ulteriori condizioni che dipendono dalla categoria del metodo individuata nel punto precedente. **Attenzione!**: se il metodo in questione lancia un'eccezione, nel blocco `ensures` dobbiamo specificare che **NON** si è verificata la condizione eccezionale.

- ***Mutator che non restituiscono nulla***

Per definizione, nel caso di esecuzione "normale" (senza eccezioni), un *mutator* modifica lo stato degli oggetti. Dobbiamo assicurarci che ciò che cambia è solo la porzione di stato modificata dal *mutator*; è necessario esplicitare nella specifica che **tutto il resto rimane invariato**. Dato che si tratta di *mutator che non restituiscono nulla* non occorre specificare il `\result`. In particolare **per ogni *observer indipendente* `obsInd`** della classe:

> **se `obsInd` permette di osservare la modifica apportata all'oggetto**, dobbiamo specificare che la modifica è avvenuta correttamente (non abbiamo modificato troppo o troppo poco). Vediamo alcune porzioni di specifica ricorrenti (ricordiamo che `obsInd` è un *place holder* per il nome dell'*observer indipendente* in questione): (*Continua nella facciata successiva*)

- **Inserire un elemento `el` di tipo `T` in un `Set<T>`**:

```
//@ requires el != null && ...;
//@ ensures (\forall T t; ; obsInd().contains(t) <==>
//@   (\old(obsInd().contains(t)) || t.equals(el))) && ...;
```

- **Rimuovere un elemento `el` di tipo `T` da una `Set<T>`**:

```
//@ requires el != null && ...;
//@ ensures (\forall T t; ; obsInd().contains(t) <==>
//@   (\old(obsInd().contains(t)) && !t.equals(el))) && ...;
```

- **Inserire un elemento `el` di tipo `T` in fondo ad una `List<T>`**:

```
//@ requires el != null && ...;
//@ ensures obsInd().size() == \old(obsInd().size() + 1) &&
//@   obsInd().get(obsInd().size() - 1).equals(el) &&
//@   (\forall int i; 0 <= i && i < \old(obsInd().size());
//@     obsInd().get(i).equals(\old(obsInd().get(i)))) && ...;
```

- **Inserire un elemento `el` di tipo `T` in una `List<T>` ordinata secondo un certo ordine**:

`mou(t1, t2)` (`mou` sta per "minore o uguale") è un *placeholder* per l'espressione che restituisce `true` sse `t1` deve precedere `t2` nella lista.
Attenzione!: questa porzione di specifica permette l'inserimento di `el` anche se è già presente nella lista, quindi non va bene se nella specifica informale viene esplicitamente detto che la lista non ammette l'inserimento di duplicati.

```
//@ requires el != null && ...;
//@ ensures obsInd().size() == \old(obsInd().size() + 1) &&
//@   (\exists int i; 0 <= i && i < obsInd().size();
//@     obsInd().get(i).equals(el) &&
//@     (\forall int j; 0 <= j && j < i;
//@       obsInd().get(j).equals(\old(obsInd().get(j))) &&
//@       mou(obsInd().get(j), obsInd().get(i))) &&
//@     (\forall int j; i < j && j < obsInd().size();
//@       obsInd().get(j).equals(\old(obsInd().get(j - 1))) &&
//@       mou(obsInd().get(i), obsInd().get(j)))) && ...;
```

- **Rimuovere un elemento dal fondo di una `List<T>`**:

Attenzione!: se la lista non contiene elementi, questa specifica ci dice che la lista rimarrà vuota (vedi la parte `\old(obsInd().size() == 0)`). Se nell'esercizio venisse richiesto di lanciare un'eccezione se la lista è vuota, occorre adattare la specifica di conseguenza.

```
//@ ensures (\old(obsInd().size() == 0) ==> obsInd().size() == 0) &&
//@   (\old(obsInd().size() > 0) ==>
//@     obsInd().size() == \old(obsInd().size() - 1) &&
//@     (\forall int i; 0 <= i && i < obsInd().size();
//@       obsInd().get(i).equals(\old(obsInd().get(i))))) && ...;
```

- **Rimuovere un elemento `el` di tipo `T` da una `List<T>`**:

Attenzione!: se `el` non fa parte della lista, questa specifica ci dice che la lista rimarrà invariata (vedi la parte `!obsInd().contains(el) ==> ...`). Se nell'esercizio venisse richiesto di lanciare un'eccezione se `el` non fa parte della lista, occorre adattare la specifica di conseguenza.

```
//@ requires el != null && ...;
//@ ensures (obsInd().contains(el) ==>
//@   obsInd().size() == \old(obsInd().size() - 1) &&
//@   (\exists int i; 0 <= i && i < \old(obsInd().size());
//@     \old(obsInd().get(i).equals(el)) &&
//@     (\forall int j; 0 <= j && j < i;
//@       obsInd().get(j).equals(\old(obsInd().get(j)))) &&
//@     (\forall int j; i <= j && j < obsInd().size();
//@       obsInd().get(j).equals(\old(obsInd().get(j + 1))))) &&
//@   (!obsInd().contains(el) ==>
//@     <specificare che una lista rimane invariata (vedi dopo)>) &&...;
```

altrimenti, se `obsInd` non permette di osservare la modifica apportata all'oggetto, dobbiamo specificare che la porzione di stato osservata è rimasta invariata. Vediamo alcune porzioni di specifica ricorrenti:

- **Specificare che un `Set<T>` rimane invariato**:

```
//@ ensures obsInd().size() == \old(obsInd().size()) &&
//@   obsInd().containsAll(\old(obsInd())) && ...;
```

- **Specificare che una `List<T>` rimane invariata**:

```
//@ ensures obsInd().size() == \old(obsInd().size()) &&
//@   (\forall int i; 0 <= i && i < obsInd().size();
//@     obsInd().get(i).equals(\old(obsInd().get(i)))) && ...;
```

- ***Observer puri***

Gli *observer* **puri** (esplicitati attraverso la keyword `/* @ pure @ */`) per definizione non possono modificare lo stato degli oggetti, quindi non è necessario specificare in JML che lo stato dell'oggetto rimane invariato (lo si considera sottinteso). Dobbiamo focalizzarci sullo specificare correttamente `\result` (gli *observer* per definizione **NON** sono `void`). Solitamente se il tipo restituito è un riferimento, bisogna specificare che non sia `null`: `\result != null`. Notiamo che, se in un TdE ci viene chiesto di specificare un *observer*, questo allora non sarà un *observer indipendente* (in quanto può essere specificato in funzione di altri *observer*). Quindi per caratterizzare `\result` **faremo riferimento alla specifica informale** e **sfrutteremo gli *observer indipendenti***. Vediamo alcune porzioni di specifica ricorrenti:

  - **`\result` è un `Set<T>` di elementi che soddisfano una determinata proprietà** (la proprietà sarà necessariamente osservabile tramite gli *observer indipendenti*):

```
//@ requires ...;
//@ ensures \result != null &&
//@   !\result.contains(null) &&
//@   (\forall T t; ;
//@     \result.contains(t) <==>
//@     <condizione sugli
//@      observer indipendenti>) && ...;
```

  - **`\result` è una `List<T>` di elementi che soddisfano una determinata proprietà**:

Come prima solo che dobbiamo specificare che nella `List` non vi sono duplicati (per i `Set` è sempre garantito)

```
//@ requires ...;
//@ ensures \result != null &&
//@   !\result.contains(null) &&
//@   (\forall T t; ;
//@     \result.contains(t) <==>
//@     <condizione sugli
//@      observer indipendenti>) &&
//@   (\forall int i; 0 <= i &&
//@     i < \result.size() - 1;
//@     (\forall int j; i < j &&
//@       j < \result.size();
//@       !\result.get(i).
//@       equals(\result.get(j)))) && ...;
```

- ***Mutator che restituiscono qualcosa***

Osserviamo che per realizzare qualsiasi *mutator che restituisce qualcosa* è sufficiente invocare prima un *observer* che ricava il risultato da restituire (facendo riferimento allo stato dell'oggetto prima della modifica ed ai parametri in input, che costituiscono tutto il contenuto informativo a nostra disposizione), salvare ciò che restituisce, invocare un *mutator che non restituisce nulla* che modifica l'oggetto secondo la specifica ed infine restituire il risultato salvato. Quindi per specificare un *mutator che restituisce qualcosa* sarà sufficiente fare riferimento alle tecniche per la specifica relative a *mutator che non restituiscono nulla* e *observer puri* e mettere in `&&` le postcondizioni ottenute (facendo sempre attenzione a ciò che si ottiene).

- ***Creator***

I *creator* creano un oggetto "dal nulla" (tutti i *creator* sono costruttori, non vale il viceversa), quindi l'oggetto creato non ha uno stato precedente all'invocazione del metodo. Per questo motivo, a differenza dei *mutator* non dobbiamo specificare ciò che cambia o ciò che rimane invariato al termine dell'esecuzione. Ci basta che lo stato osservato sia compatibile con la specifica e con i parametri in input. Per farlo è sufficiente, **per ogni *observer indipendente***, esplicitare che valore restituisce. Nell'esempio `InsiemeDiInteri`, l'unico *observer indipendente* è `appartiene`. Da specifica il costruttore di `InsiemeDiInteri`, che è anche un *creator*, inizializza un insieme vuoto, quindi la specifica corrispondente sarà:

```
public class InsiemeDiInteri {
  ...
  //@ ensures !(\exists int x; ;
  //@   appartiene(x));
  public InsiemeDiInteri();
  ...
}
```

In realtà in questo caso possiamo usare un *observer dipendente* per scrivere una specifica equivalente in maniera più compatta:

```
public class InsiemeDiInteri {
  ...
  //@ ensures cardinalita() == 0;
  public InsiemeDiInteri();
  ...
}
```

Solitamente nei TdE non viene richiesto di specificare dei *creator*.

- ***Producer in classi pure (immutabili)***

Dato che la classe che stiamo specificando è **pura** (`public /* @ pure @ */ class ...`), come nel caso degli *observer puri* non è necessario esplicitare la relazione tra lo stato corrente e quello passato (dire ciò che cambia e ciò che non cambia); rimane sempre tutto invariato. Stiamo costruendo un nuovo oggetto della stessa classe che stiamo specificando, come nel caso dei *creator* ne specificheremo lo stato iniziale esplicitando il valore restituito da **ogni *observer indipendente***. **A differenza dei *creator*** però, **l'oggetto che si trova nello stato iniziale specificato è `\result`**, **NON** `this`. Un'ulteriore differenza è che lo stato iniziale di `\result` non dipende solo dai parametri di input del metodo, ma anche dallo stato dell'oggetto sul quale invochiamo il *producer* (`this`). Solitamente nei TdE non viene richiesto di specificare dei *producer*.

Ricavare la postcondizione eccezionale (`signals`)

- ***Mutator che non restituiscono nulla***

Quando si verifica un'eccezione durante l'esecuzione di un *mutator*, ci si aspetta che lo stato dell'oggetto su cui abbiamo invocato il metodo rimanga invariato. Quindi, nella postcondizione nel caso eccezionale, **oltre** a esplicitare cosa ha provocato l'eccezione (facendo riferimento alla specifica informale), specificheremo che lo stato dell'oggetto è rimasto invariato: **per ogni *observer indipendente*** il valore restituito da tale *observer* è lo stesso di prima che eseguissimo il *mutator*. Possiamo sfruttare le porzioni di specifica ricorrenti introdotte nel paragrafo precedente (`Set` che non varia, `List` che non varia, ...).

- ***Observer puri***

Gli *observer puri* non modificano lo stato dell'oggetto su cui vengono invocati. Nella postcondizione eccezionale è sufficiente esplicitare cosa ha provocato l'eccezione, facendo riferimento alla specifica informale.

- ***Mutator che restituiscono qualcosa***

Dato che il metodo ha lanciato un'eccezione, il fatto che restituisse qualcosa è irrilevante: ci si comporta come nel caso in cui vogliamo specificare la postcondizione eccezionale di un *mutator che non restituisce nulla*.

- ***Creator***

Quando un *creator* fallisce (lancia un'eccezione) non c'è nessuno stato che possiamo osservare: l'oggetto non è stato costruito. È sufficiente specificare cosa ha provocato l'eccezione, facendo riferimento alla specifica informale.

- ***Producer in classi pure (immutabili)***

Il fatto che la classe sia **pura** implica che lo stato dell'oggetto non possa cambiare: come nel caso degli *observer puri* è sufficiente esplicitare cosa ha provocato l'eccezione, facendo riferimento alla specifica informale.

Ricavare l'invariante di rappresentazione

Se il `rep` è di tipo riferimento solitamente occorre specificare `rep != null`.

Se il `rep` è una collezione di oggetti solitamente occorre specificare `!rep.contains(null)` (nel caso in cui `rep` sia una `Map`: `!rep.containsKey(null) && !rep.containsValue(null)`).

Se il `rep` è una collezione di collezioni di oggetti solitamente occorre specificare che ciascuna "sottocollezione" non sia vuota e non contenga `null`. Nel caso in cui il `rep` sia una `Map<K, V>` dove `V` estende `Collection<T>`:

```
//@ private invariant
//@   (\forall K k; rep.containsKey(k);
//@     rep.get(k).size() > 0 &&
//@     !rep.get(k).contains(null)) && ...;
```

Nel caso in cui il `rep` sia un `Set<V>` dove `V` estende `Collection<T>`:

```
//@ private invariant
//@   (\forall V v; rep.contains(v);
//@     v.size() > 0 && !v.contains(null))
//@   && ...;
```

Nel caso in cui il `rep` sia una `List<V>` dove `V` estende `Collection<T>`:

```
//@ private invariant
//@   (\forall int i; 0 <= i && i <rep.size();
//@     rep.get(i).size() > 0 &&
//@     !rep.get(i).contains(null)) && ...;
```

Ricavare la funzione di astrazione

Per ricavare la funzione di astrazione è importante ricordare che è possibile definire completamente lo stato astratto di un oggetto attraverso gli *observer indipendenti*. Quindi è sufficiente definire il valore restituito da ciascun *observer indipendente* in funzione del `rep`.

Esercizio (TdE del 2022-02-07 esercizio 1 - punto a, b e c)

Vediamo come risolvere un esercizio preso da un TdE applicando i *trucchi* discussi nel paragrafo precedente.

Testo dell'esercizio

Si consideri la classe Java `JobsDB` per la gestione di domande e offerte di lavoro (classi `JobRequest` e `JobOffer`).

```
public class JobsDB {
  // Ritorna l'elenco delle richieste salvate.
  public /*@ pure @*/ Set<JobRequest>
    getRequests();
  // Ritorna l'elenco delle offerte salvate
  // in ordine cronologico dalla meno alla
  // piu' recente.
  public /*@ pure @*/ List<JobOffer>
    getOffers();
  // Aggiunge una richiesta di lavoro.
  // Lancia una DuplicateException se la
  // richiesta e' gia' presente.
  public void addRequest(JobRequest req)
    throws DuplicateException;
  // Aggiunge un'offerta di lavoro e
  // ritorna l'insieme di tutte le richieste
  // di lavoro compatibili con l'offerta
  // (senza rimuoverle)
  public Set<JobRequest>
    addOffer(JobOffer offer);
  // Ricerca un'offerta di lavoro compatibile
  // on la richiesta req. Se esiste almeno
  // un'offerta di lavoro compatibile con la
  // richiesta, ritorna la prima offerta
  // compatibile (meno recente) e la rimuove.
  // Altrimenti, ritorna null.
  public JobOffer getOffer(JobRequest req);
}
public /*@ pure @*/ class JobRequest {
  // Ritorna true se e solo se l'offerta
  // e' compatibile con la richiesta.
  public boolean matches(JobOffer offer);
... }
```

Gli *observer indipendenti* di `JobsDB` sono `getRequests` e `getOffers`.

Domanda a)

Si specifichi in JML il metodo `addOffer`.

`addOffer` è un *mutator che restituisce qualcosa*. Dai *trucchi* sulla precondizione, dato che il parametro offer è di tipo riferimento, imponiamo nella precondizione `offer != null`.

```
//@ requires offer != null;
...
```

Ricaviamo la postcondizione normale: l'*observer indipendente* che permette di osservare la modifica è `getOffers`; possiamo riutilizzare la porzione di specifica ricorrente *inserire un elemento `el` di tipo `T` in fondo ad una `List<T>`*:

```
...
//@ ensures getOffers().size() == \old(getOffers().size() + 1) &&
//@   getOffers().get(getOffers().size() - 1).equals(offer) &&
//@   (\forall int i; 0 <= i && i < \old(getOffers().size());
//@     getOffers().get(i).equals(\old(getOffers().get(i)))) &&
...
```

Ma NON abbiamo ancora finito!: dobbiamo specificare, per ogni altro *observer indipendente* (`getRequests`), che tutto è rimasto invariato; possiamo riutilizzare la porzione di specifica ricorrente *specificare che un Set rimane invariato*:

```
...
//@   getRequests().size() == \old(getRequests().size()) &&
//@   getRequests().containsAll(\old(getRequests())) &&
...
```

Abbiamo finito di specificare la "parte" *mutator*, ora dobbiamo specificare la "parte" *observer* (ricordiamo che `addOffer` è un *mutator che restituisce qualcosa*); possiamo riutilizzare la porzione di specifica *\result è un Set di elementi che soddisfano una determinata proprietà* (la proprietà è osservabile tramite `getRequests`):

```
...
//@   \result != null && !\result.contains(null)
//@   (\forall JobRequest r; ;
//@     \result.contains(r) <==> (getRequests().contains(r) &&
//@     r.matches(offer)));
public Set<JobRequests> addOffer(JobOffer offer);
```

(In questo caso abbiamo già specificato che `getRequests` non varia, possiamo scrivere la specifica come se stessimo trattando un normale *observer*).

Dato che il metodo non lancia eccezioni non sarà necessario specificare il blocco `signals`.

Domanda b)

Si specifichi in JML il metodo `getOffer`.

Anche `getOffer` è un mutator che restituisce qualcosa, procediamo come prima scrivendo la specifica secondo il seguente schema:

```
//@ requires <req non nullo>
//@ ensures <esiste un'offerta compatibile> ?
//@   (* specifica della parte mutator *) &&
//@     <rimuovere la prima offerta compatibile dalla lista getOffers>&&
//@     <il set getRequests rimane invariato> &&
//@   (* specifica della parte observer *) &&
//@     <\result è l'elemento rimosso dalla lista> :
//@   (* specifica della parte mutator *) &&
//@     <la lista getOffer rimane invariata> &&
//@     <il set getRequests rimane invariato> &&
//@   (* specifica della parte observer *)
//@     <\result è nullo>
```

Possiamo eliminare le ripetizioni come segue:

```
//@ requires <req non nullo>
//@ ensures (<esiste un'offerta compatibile> ?
//@   <rimuovere la prima offerta compatibile dalla lista getOffers> &&
//@   <\result è l'elemento rimosso dalla lista> :
//@   <la lista getOffer rimane invariata> &&
//@   <\result è nullo>) &&
//@ <il set getRequests rimane invariato>;
```

Siamo pronti per scrivere la specifica (*Continua nella facciata successiva*):

```
//@ requires req != null;
//@ ensures ((\exists JobOffer o; \old(getOffers().contains(o));
//@   req.matches(o)) ?
//@     (getOffers().size() == \old(getOffers().size() - 1) &&
//@     (\exists int i; 0 <= i && i < \old(getOffers().size());
//@       \old(req.matches(getOffers().get(i))) &&
//@       (\forall int j; 0 <= j && j < i;
//@         getOffers().get(j).equals(\old(getOffers().get(j))) &&
//@         !req.matches(getOffers().get(j))) &&
//@       (\forall int j; i <= j && j < getOffers().size();
//@         getOffers().get(j).equals(\old(getOffers().get(j + 1)))) &&
//@       \result.equals(\old(getOffers().get(i))))) :
//@     (getOffers().size() == \old(getOffers().size()) &&
//@     (\forall int i; 0 <= i && i < getOffers().size();
//@       getOffers().get(i).equals(\old(getOffers().get(i)))) &&
//@     \result == null)) &&
//@   getRequests().size() == \old(getRequests().size()) &&
//@   getRequests().containsAll(\old(getRequests()));
public JobOffer getOffer(JobRequest req);
```

Nello scrivere la specifica abbiamo utilizzato alcune porzioni ricorrenti per la lista o il set che rimangono invariati ed abbiamo riadattato *rimuovere un elemento `el` di tipo `T` da una `List`<`T`>*. Risulta evidente che le tecniche di cui abbiamo parlato non permettono di ricavare la specifica in modo meccanico, aiutano solo a non dimenticare qualcosa.

Domanda c)

Si consideri un'implementazione di `JobsDB` che fa uso di un `Set` per memorizzare le richieste di lavoro e di una `List` per memorizzare le offerte di lavoro, come mostrato di seguito.

```
public class JobsDB {
  private Set<JobRequest> requests;
  private List<JobOffer> offers;
... }
```

Per tale implementazione, fornire l'invariante di rappresentazione (representation invariant, RI) e la funzione di astrazione (abstraction function, AF).

Dalle tecniche per ricavare l'RI deduciamo `requests` e `offers` non nulli e `requests` e `offers` non contengono `null`. (Non viene esplicitamente detto che `offers` non contiene duplicati ed è anche corretto semanticamente: ci possono essere più offerte uguali in momenti diversi)

```
// RI:
//@ private invariant requests != null && !requests.contains(null) &&
//@   offers != null && !offers.contains(null);
```

Per definire l'AF è sufficiente specificare gli *observer indipendenti* `getRequests` e `getOffers` tramite `requests` e `offers`:

```
// AF:
//@ private invariant
//@   getRequests().size() == requests.size() &&
//@   getRequests().containsAll(requests) &&
//@   getOffers().size() == offers.size() &&
//@   (\forall int i; 0 <= i && i < offers.size();
//@     getOffers().get(i).equals(offers.get(i)));
```

(È molto simile alla specifica di una lista o di un set che non variano).

Principio di sostituzione

Secondo il principio *open-closed* un modulo (che in Java coincide con una classe) dovrebbe essere *chiuso* rispetto alla modifica ed *aperto* rispetto all'estensione. Cioè se vogliamo arricchire il comportamento di un certo modulo, non possiamo modificarne la specifica, bensì dobbiamo estenderlo con un sottomodulo che aggiunge nuove funzionalità.

In Java, per realizzare l'estensione ricorriamo al meccanismo di ereditarietà. Dobbiamo però fare attenzione: **NON** tutte le sottoclassi ottenute tramite ereditarietà realizzano un'estensione della superclasse. In particolare, perchè una sottoclasse `B` **estenda** la corrispondente superclasse `A`, gli oggetti di `B` devono essere *sostituibili* a quelli di `A`; nel senso che gli oggetti di `B` possono essere utilizzati da un utente come oggetti della classe `A`, senza notare alcuna differenza.

Per formalizzare questo concetto di *sostituibilità*, introduciamo il **principio di sostituzione di Liskov**: *"Tutti gli oggetti della sottoclasse devono soddisfare il contratto della superclasse"*, ma il contratto può essere esteso per coprire ulteriori casi.

Consideriamo l'esempio `InsiemeDiInteri` introdotto nel paragrafo *Esempio* nel capitolo *JML*. Aggiungiamo alla classe `InsiemeDiInteri` un `Iterator` che permette di iterare sugli elementi dell'insieme senza garantire un ordine preciso (scriviamo solo la specifica):

```
public class InsiemeDiInteri {
  ...
  //@ ensures (* restituisce un iteratore che
  //@   itera su tutti e soli gli elementi
  //@   dell'insieme, ognuno visitato una
  //@   volta, in un ordine qualsiasi *);
  public Iterator<Integer> iterator();
  ...
}
```

Supponiamo che un utente di `InsiemeDiInteri` voglia implementare il metodo `traccia` che calcola la somma di tutti gli elementi appartenenti all'insieme (ciascuno considerato nella somma una ed una sola volta):

```
public static int traccia(InsiemeDiInteri s) {
  int somma = 0;
  for (Iterator<Integer> iter = s.iterator();
    iter.hasNext(); ) {
    somma += iter.next();
  }
  return somma;
}
```

Consideriamo le seguenti sottoclassi che ereditano da `InsiemeDiInteri`:

```
public class InsiemeDiInteriIteratorOrdinato
  extends InsiemeDiInteri {
  @Override
  //@ ensures (* restituisce un iteratore che
  //@   itera su tutti e soli gli elementi
  //@   dell'insieme, ognuno visitato una
  //@   volta, in ordine crescente *);
  public Iterator<Integer> iterator();
}
```

e

```
public class InsiemeDiInteriIteratorPositivi
  extends InsiemeDiInteri {
  @Override
  //@ ensures (* restituisce un iteratore che
  //@   itera su tutti e soli gli elementi
  //@   positivi dell'insieme, ognuno visitato
  //@   una volta,
  //@   in un ordine qualsiasi *);
  public Iterator<Integer> iterator();
}
```

Il metodo `traccia` continua a funzionare sugli oggetti della classe `InsiemeDiInteriIteratorOrdinato` (dato che vale la proprietà commutativa della somma). In particolare la ridefinizione di `iterator` di questa sottoclasse continua a soddisfare il contratto della superclasse (dato che la superclasse ammette che gli elementi vengano visitati in un ordine **qualsiasi**). Al contrario quando richiamiamo `traccia` con un oggetto della classe `InsiemeDiInteriIteratorPositivi`, ciò che viene calcolato è la somma dei soli elementi positivi dell'insieme e non la *traccia* dell'insieme. Infatti la ridefinizione di `iterator` in `InsiemeDiInteriIteratorPositivi` viola il contratto della superclasse: **NON** itera su tutti e soli gli elementi dell'insieme, ma solo sui positivi. In conclusione `InsiemeDiInteriIteratorOrdinato` soddisfa il principio di sostituzione, `InsiemeDiInteriIteratorPositivi` **NO**.

Regole per soddisfare il principio di sostituzione

Per assicurarci che il contratto di una sottoclasse sia compatibile con quello della superclasse (e quindi che la sottoclasse soddisfi il principio di sostituzione) è sufficiente verificare che valgano le tre *regole* che seguono:

- *signature rule*: una sottoclasse deve disporre di tutti i metodi della superclasse e le firme dei metodi della sottoclasse devono essere **compatibili** con quelle dei metodi della superclasse
- *method rule*: le chiamate ad un metodo della sottoclasse si devono "comportare" come le chiamate al metodo corrispondente della superclasse
- *property rule*: una sottoclasse deve preservare tutte le proprietà astratte (vedi capitolo sul JML) degli oggetti della superclasse

La *signature rule* garantisce che la sintassi con cui operiamo sulla sottoclasse sia compatibile con la sintassi della superclasse. La *method rule* garantisce che il contratto (inteso come la specifica) di ciascun metodo *ereditato* sia compatibile con il contratto del metodo *originale*. In particolare un'estensione si dice **pura** se non cambia la specifica (aggiunge solo dei metodi). La *property rule* garantisce che la specifica della sottoclasse nella sua interezza sia compatibile con quella della classe originale.

Signature rule

La *signature rule* è verificabile **staticamente** dal compilatore di Java.

In particolare la *signature rule* "implementata" in Java è: una sottoclasse deve avere tutti i metodi della superclasse e la firma dei metodi della sottoclasse deve essere **identica** a quella dei metodi corrispondenti della superclasse. Un metodo della sottoclasse può però avere meno eccezioni di quello della superclasse (ricordiamo che in Java la firma di un metodo è costituita dal suo nome e dalla lista dei parametri, ma non include il tipo restituito, quindi non possiamo dichiarare nella stessa classe due metodi che hanno lo stesso nome e la stessa lista di parametri anche se restituiscono un tipo diverso). Notiamo che la *signature rule* in Java è **più restrittiva** di quella enunciata nel caso generale, infatti esistono metodi che hanno firma **compatibile** ma **NON identica**. Per dimostrarlo occorre introdurre i concetti di controvarianza e covarianza.

Controvarianza e covarianza

L'esempio che segue non è valido in Java, dato che ci serve per illustrare i concetti di **controvarianza** e **covarianza** che non sono entrambi validi nel linguaggio. Consideriamo una superclasse `A` che definisce il metodo `f : PA -> RA` (dove `PA` è la classe dei parametri di `f` e `RA` è la classe degli oggetti restituiti da `f`). Sia `D` una sottoclasse di `A` che **ridefinisce** il metodo `f` come `f : PD -> RD`. Consideriamo uno snippet di codice che utilizza un oggetto della classe `A` (dato che non facciamo riferimento ad un linguaggio in particolare useremo una sintassi inventata):

```
method void usaA(A a) {
  ...
  RA rA = a.f(pA);
  ...
}
```

Supponiamo di passare come parametro in `usaA` un oggetto `d` della classe `D`. Perchè la *signature rule* sia verificata, l'oggetto `pA`, che ha tipo dinamico `PA` deve poter essere accettato come parametro anche da `D.f`, che si aspetta un parametro di tipo `PD`. Quindi la classe `PD` può coincidere con la classe `PA` o può essere un **supertipo** della classe `PA`. Il fatto che se `PD` è una superclasse di `PA`, la *signature rule* è ancora soddisfatta, si dice **controvarianza dei parametri di input**. La controvarianza dei parametri di input **NON** è supportata in Java, dove, per fare un override, è necessario che `PD` **coincida** con `PA`.

Consideriamo ora il valore restituito da `a.f` (sempre nel caso in cui abbiamo passato ad `usaA` un oggetto `d` della classe `D`). `a.f` restituirà un oggetto della classe `RD`, che, perché la *signature rule* sia verificata, deve poter essere assegnato ad un oggetto di tipo `RA` come l'utente si aspetta. Quindi `RD` può coincidere con `RA` oppure può essere una sottoclasse di `RA`. Il fatto che se `RD` è una sottoclasse di `RA`, la *signature rule* è ancora soddisfatta, si dice **covarianza dei risultati**. La covarianza dei risultati è ammissibile in Java a partire da Java 5.

Method rule

Perché la chiamata di un metodo di una sottoclasse abbia lo stesso effetto della chiamata del metodo della classe originale, è sufficiente che il metodo *ereditato* soddisfi la specifica del metodo *originale*. Questo **NON** è verificabile staticamente dal compilatore. In termini non formali un metodo *ereditato* soddisfa la specifica del metodo *originale* se ha una precondizione più *debole* (richiede meno) ed una postcondizione più *forte* (promette di più). Formalizziamolo.

Condizioni *forti* e *deboli*

Una condizione è *più forte* di un'altra se è *vera "in meno casi"*. Questo concetto si formalizza in logica attraverso l'implicazione: una condizione `a` è ***più forte*** di una condizione `b` se è sempre vera la condizione `a ==> b`. Questo introduce un ordinamento (non totale) delle formule logiche dalla *più forte* alla *più debole*. In particolare `false` è la condizione *più forte* in assoluto e `true` la *più debole* in assoluto.

Vediamo l'**effetto degli operatori logici** sulla `forza` delle condizioni:

- `||` *indebolisce*: `a ==> a || b` (cioè `a || b` è *più debole* di `a`)
- `&&` *rafforza*: `a && b ==> a` (cioè `a && b` è *più forte* di `a`)
- `==>` *indebolisce*: `a ==> (b ==> a)`, si deduce ricordando che `b ==> a` equivale ad `!b || a`
- se *rafforziamo* la premessa *indeboliamo* l'implicazione: supponiamo che `c` sia *più forte* di `a`, allora è sempre vero `c ==> a`. Allora, se `a ==> b` è vero, per transitività `c ==> b` è vero, cioè abbiamo dimostrato che `(a ==> b) ==> (c ==> b)` e cioè che `a ==> b` è *più forte* di `c ==> b`.

Precondizione *più debole*

Se la precondizione del metodo *ereditato* è più debole di quella del metodo *originale* allora se è vera la precondizione del metodo *originale* è anche vera quella del metodo *ereditato* e cioè possiamo richiamare il metodo *ereditato* in tutti i casi in cui potevano richiamare quello *originale*; proprio come ci aspetteremmo! Quindi condizione **necessaria** perchè valga la *method rule* è la ***precondition rule***: detta `pre_sub` la precondizione del metodo *ereditato* e `pre_super` la precondizione di quello *originale*, `pre_sub` è *più debole* di `pre_super` e cioè `pre_super ==> pre_sub` è sempre vera.

Se `pre_sub` non fosse *più debole* di `pre_super` esisterebbe un caso in cui *pre_sub* è falsa e *pre_super* è vera. L'utente richiamando il metodo in quel caso (che soddisfa la precondizione del metodo *originale* e quindi risulta un caso valido di input) otterrebbe un risultato inaspettato (dato che tale caso viola la precondizione del metodo *ereditato* che è quello che viene effettivamente invocato).

Postcondizione *più forte*

Alla fine dell'esecuzione del metodo *ereditato* ci aspettiamo che la postcondizione del metodo *originale*, detta `post_super` sia soddisfatta. L'utente, nella sua implementazione, infatti fa riferimento solo alla postcondizione del metodo *originale* e non deve preoccuparsi se in realtà ad essere invocato è il metodo *ereditato*. Quindi, detta `post_sub` la postcondizione del metodo *ereditato*, una condizione **sufficiente** (assumendo che la *precondition rule* sia stata rispettata) perchè valga la *method rule* è che `post_sub` sia più forte di `post_super`, ovvero `post_sub ==> post_super`. In questo modo, dato che il metodo *ereditato* deve soddisfare la sua postcondizione, soddisferà automaticamente anche le postcondizione del metodo *originale*.

In realtà questa condizione è più stringente del necessario. L'utente invocherà il metodo solo nei casi che soddisfano la precondizone del metodo ***originale*** ed abbiamo visto che in generale la precondizione del metodo *ereditato* può "ammettere più casi". Quindi ci basta che valga la ***full postcondition rule*** cioè che `post_sub` implichi che `post_super` sia vera se `pre_super` era vera al momento della chiamata del metodo, che in formule diventa: `post_sub ==> (\old(pre_super) ==> post_super)`. Sulle slide la *full postcondition rule* è espressa come `(\old(pre_super) && post_sub) ==> post_super` (in realtà senza `\old` ma è più corretto aggiungerlo); le due formule sono equivalenti, infatti `post_sub ==> (\old(pre_super) ==> post_super) = !post_sub || (!\old(pre_super) || post_super) = (!\old(pre_super) || !post_sub) || post_super = !(!\old(pre_super) || !post_sub) ==> post_super = (\old(pre_super) && post_sub) ==> post_super`.

Specificare metodi che ne estendono altri in JML

Per specificare un metodo che ne estende un altro in JML si utilizza la keyword `also` che si inserisce nella specifica in questo modo:

```
//@ also
//@ requires pre_ext;
//@ ensures post_ext;
```

Notiamo che in `requires` ed `ensures` compaiono `pre_ext` e `post_ext` invece di `pre_sub` e `post_sub`. Questo perchè la semantica della specifica con `also` si traduce in:

```
//@ requires pre_super || pre_ext;
//@ ensures (\old(pre_super) ==> post_super)
//@   && (\old(pre_ext) ==> post_ext);
```

Quindi `pre_sub = pre_super || pre_ext` e `post_sub = (\old(pre_super) ==> post_super) && (\old(pre_ext) ==> post_ext)`.

Notiamo che `pre_sub` è più debole di `pre_super` e che, dato che `&&` rafforza, vale `post_sub ==> (\old(pre_super) ==> post_super)`. Quindi i metodi ottenuti per estensione, se specificati attraverso JML, **soddisfano sempre la *method rule*** dato che valgono la ***precondition rule*** e la ***full postcondition rule***.

Osserviamo che specificando i metodi con `also`:

- **Non possiamo *rafforzare* la precondizione**: supponiamo `pre_ext` *più forte* di `pre_super`, allora `pre_ext ==> pre_super` è sempre vera, allora `!pre_ext || pre_super = true`, allora `pre_ext && !pre_super = false`, allora `pre_sub = pre_super || pre_ext = pre_super || pre_ext && true = pre_super || pre_ext && (pre_super || !pre_super) = pre_super || pre_ext && pre_super || pre_ext && !pre_super = pre_super && true || pre_super && pre_ext || false = pre_super && (true || pre_ext) = pre_super && true = pre_super`
- **Non possiamo *indebolire* la postcondizione**: supponiamo `post_ext` *più debole* di `post_super`, allora `post_super ==> post_ext` è sempre vera, allora, nell'ipotesi che `\old(pre_super)` sia verificata, `post_sub = (true ==> post_super) && (\old(pre_ext) ==> post_ext) = post_super && (\old(pre_ext) ==> post_ext) = post_super` (se `post_super` vale `false` allora `post_sub = post_super && ...` vale `false`, se `post_super` vale `true`, allora `post_ext` vale `true`, allora anche `\old(pre_ext) ==> post_ext` vale `true`, allora `post_sub` vale `true`).
- **Se volessimo semplicemente *rafforzare* la postcondizione è sufficiente che `pre_ext = pre_super` e che `post_ext` sia *più forte* di `post_super`**: `pre_sub = pre_super || pre_ext = pre_super || pre_super = pre_super` e, supponendo che `\old(pre_super) = \old(pre_ext)` sia verificata, `post_sub = (\old(pre_super) ==> post_super) && (\old(pre_ext) ==> post_ext) = (true ==> post_super) && (true ==> post_ext) = post_super && post_ext = post_ext` (se `post_ext` vale `false` allora `post_sub = ... && post_ext` vale `false`, se `post_ext` vale `true` allora anche `post_super` vale `true`, allora `post_sub` vale `true`)

Eliminare le eccezioni

Abbiamo già spiegato che è possibile rimuovere alcune delle eccezioni quando si fa un override di un metodo, senza violare la *signature rule*. Dobbiamo però fare molta attenzione che la *method rule* sia ancora soddisfatta. Consideriamo la seguente classe:

```
public class InsiemeDiInteriEccezDuplicati {
  ...
  //@ ensures !\old(appartiene(x)) &&
  //@   appartiene(x) && ...;
  //@ signals (EccezioneDuplicato e)
  //@   \old(appartiene(x));
  public void inserisci(int x)
    throws EccezioneDuplicato;
  ...
}
```

che restituisce un'eccezione quando si prova ad inserire un elemento già presente nell'insieme. Per ipotesi

```
public class InsiemeDiInteri
  extends InsiemeDiInteriEccezDuplicati {
  ...
  //@ ensures appartiene(x) &&
  //@ cardinalita() == \old(cardinalita() +
  //@ (appartiene(x) ? 0 : 1)) &&
  //@ (\forall int y; \old(appartiene(y));
  //@ appartiene(y));
  public void inserisci(int x);
  ...
}
```

supponiamo che `InsiemeDiInteri` sia una **sottoclasse** di `InsiemeDiInteriEccezDuplicati` e che ridefinisca il metodo `inserisci` secondo la specifica che abbiamo introdotto originariamente (nel capitolo *JML*). Secondo ciò che abbiamo detto `InsiemeDiInteri` soddisfa la *signature rule*. Un utente di `InsiemeDiInteriEccezDuplicati` però, si aspetta di ricevere un'eccezione se inserisce un elemento due volte di seguito nell'insieme, ma questo **NON** avviene se il tipo dinamico dell'insieme è `InsiemeDiInteri`.

Quindi, in questo esempio, `InsiemeDiInteri` **viola** la *method rule* e **NON** rispetta il *principio di sostituzione*.

Property rule

Una classe gode di due macrocategorie di proprietà astratte: le proprietà *invarianti* e le proprietà *evolutive* (vedi capitolo sul *JML*). Le prime possono essere esplicitate in JML attraverso il costrutto `public invariant`, ma questo non avviene sempre; al contrario le seconde sono deducibili solo considerando le specifiche di tutti i metodi della classe, nel loro complesso. Perchè una **sottoclasse** soddisfi la *property rule*, deve conservare le proprietà astratte della classe che estende, **per tutti gli stati astratti osservabili tramite gli osservatori della classe *originale***. Non è infatti necessario che le proprietà astratte della classe *originale* valgano anche per la "porzione" di stato astratto della classe *derivata* che è osservabile tramite dei metodi presenti solo in quest'ultima.

Ad esempio se una classe *immutabile* `A` viene estesa da una classe `B` definita come segue:

```
public class B extends A {
  private int b;

  public void setB(int b) { this.b = b; }

  public int getB() { return b; }
}
```

Nonostante `B` sia chiaramente *mutabile*, la "porzione" del suo stato astratto che muta (l'attributo `b`) non è osservabile con gli *observer* di `A`: quindi `B` soddisfa la *property rule* (non viola la proprietà evolutiva di *immutabilità* rispetto allo stato osservabile dagli utenti di `A`).

A differenza della *signature rule* e della *method rule*, nulla garantisce che la *property rule* sia stata rispettata, quindi è necessario valutarlo volta per volta in base alle specifiche di tutti i metodi della classe.

Esempio di violazione di una proprietà invariante

Consideriamo la classe `InsiemeDiInteriPieno`, ovvero un `InsiemeDiInteri` che non è mai vuoto:

```
public class InsiemeDiInteriPieno {
  // OVERVIEW: Insieme di interi illimitato
  //   e mutabile con almeno un elemento

  //@ public invariant cardinalita() >= 1;

  //@ ensures (* restituisce un
  //@   InsiemeDiInteriPieno che rappresenta
  //@   l'insieme { i } *);
  public InsiemeDiInteriPieno(int i);

  //@ ensures (\old(cardinalita() >= 2) ==>
  //@   !appartiene(x)) &&
  //@   ((\old(cardinalita() == 1)) ==>
  //@   (* l'insieme rimane invariato *));
  public void rimuoviSeNonVuoto(int x);
}
```

Supponiamo che `InsiemeDiInteri` sia una **sottoclasse** di `InsiemeDiInteriPieno` che aggiunge il metodo `rimuovi` così come lo avevamo definito originariamente:

```
public class InsiemeDiInteri
  extends InsiemeDiInteriPieno {

  //@ ensures !appartiene(x) &&
  //@ cardinalita() == \old(cardinalita() -
  //@   (appartiene(x) ? 1 : 0)) &&
  //@ (\forall int y; appartiene(y);
  //@   \old(appartiene(y)));
  public void rimuovi(int x);
}
```

Dato che `InsiemeDiInteri` non ridefinisce alcun metodo di `InsiemeDiInteriPieno`, la *signature rule* e la *method rule* risultano soddisfatte. Grazie però al nuovo metodo `rimuovi` possiamo violare l'invariante pubblico di `InsiemeDiInteriPieno` svuotando l'insieme. Quindi, in questo esempio, `InsiemeDiInteri` **viola** la *property rule* e di conseguenza **NON** rispetta il *principio di sostituzione*.

Esempio di violazione di una proprietà evolutiva

Consideriamo la classe `InsiemeDiInteriSenzaRimuovi` che è uguale ad `InsiemeDiInteri` ma non ha il metodo `rimuovi`. `InsiemeDiInteriSenzaRimuovi` gode della seguente proprietà *evolutiva*: *"il valore restituito da `cardinalita` nello stato prossimo è maggiore o uguale a quello restituito nello stato corrente"* (l'insieme non si "restringe"). Supponiamo che la classe `InsiemeDiInteri` estenda la classe `InsiemeDiInteriSenzaRimuovi` aggiungendo, come nell'esempio precedente, solo il metodo `rimuovi` definito come al solito. Come nell'esempio precedente la *signature rule* e la *method rule* non possono essere violate, dato che `InsiemeDiInteri` non ridefinisce alcun metodo. Chiaramente ad essere violata è la proprietà evolutiva di `InsiemeDiInteriSenzaRimuovi` e di conseguenza la *property rule*.

Programmazione concorrente

La programmazione concorrente permette di svolgere più attività (dette task) in maniera simultanea.

L'esecuzione può avvenire in parallelismo fisico (ovvero mediante l'utilizzo di più processori o un processore multi-core) o mediante la condivisione di un unico processore, seguendo le decisioni dello scheduler (sistemi time-shared).

Si può avere concorrenza:

- A livello di processi: ogni processo ha un suo spazio di memoria riservato, la comunicazione fra di essi deve avvenire mediante meccanismi di IPC (Inter-process communication). Non è argomento del corso.
- A livello di **thread**: un singolo processo può contenere numerosi thread, che condividono tutti lo stesso spazio di indirizzamento e possono comunicare mediante variabili condivise.

La libreria standard Java mette a disposizione, oltre alla primitiva di sistema Thread, anche una serie di primitive per semplificare la programmazione concorrente.

Thread

Per creare un thread, è sufficiente estendere la classe `java.lang.Thread`, fare l'override del metodo `run()`, istanziare la classe e chiamare il metodo `start()` (**NB: start, non run**):

```
class MyThread extends Thread {
  @Override
  public void run() {
    // Fai cose
  }
}
...
 // Fa partire il thread e ritorna subito
new MyThread().start();
```

Per motivi di riutilizzo del codice, viene solitamente preferito utilizzare un metodo alternativo per la creazione di un Thread, implementando invece l'interfaccia funzionale `java.lang.Runnable`:

```
class MyRunnable implements Runnable {
  @Override
  public void run() {
    // Fai cose
  }
}
...
// Fa partire il thread e ritorna subito
new Thread(new MyRunnable()).start();
```

Oppure usando i paradigmi della programmazione funzionale:

```
Runnable r = () -> {
  // Fai cose
};

new Thread(r).start();
```

O in modo ancora più compatto:

```
new Thread(() -> { /* Fai cose */ }).start();
```

Notiamo come, in entrambi i casi di creazione (estendendo `Thread` o implementando `Runnable`), il thread sia libero di accedere a qualsiasi variabile globale o passata nel costruttore, causando potenzialmente errori di interferenza se più thread accedono alle stesse variabili contemporaneamente.

Intrinsic lock

Java, di default, associa a ogni oggetto un proprio "lock", ovvero un oggetto che permette di implementare mutua esclusione tra i vari thread su sezioni di codice di interesse. Questo avviene mediante l'uso della keyword `synchronized`, specificando poi l'oggetto di cui si vuole utilizzare il lock:

```
class Account {
  private float balance;

  public Account(float balanceIniz) {
    balance = balanceIniz;
  }

  public void deposit(float money) {
    synchronized(this) {
      balance += money;
    }
  }
  public void withdraw(float money) {
    synchronized(this) {
      balance -= money;
    }
  }
}
```

L'intrinsic lock:

- mette in mutua esclusione sezioni di codice, permettendo a un solo thread alla volta l'esecuzione, sospendendo gli altri thread chiamanti fino al completamento.
- é un lock rientrante: se il thread, quando raggiunge una sezione critica, è già in possesso del lock, continuerà la sua esecuzione senza venire sospeso.
- quando viene acquisito/rilasciato, forza la sincronizzazione della cache locale del thread con la memoria centrale, garantendo che tutte le variabili lette siano aggiornate.

Notiamo come, a causa della cache locale del thread menzionata al terzo punto, sia necessario mettere in sezioni sincronizzate anche metodi che leggono solo una variabile, senza modificarla. Questo perché, sebbene il metodo stesso non la modifichi, la variabile potrebbe però essere cambiata esternamente da altri thread, causando un desync tra la cache locale e la memoria centrale.

Esiste anche una forma più breve di scrivere il codice precedente, mettendo la keyword direttamente sul metodo:

```
class Account {
  private float balance;

  public Account(float balanceIniz) {
    balance = balanceIniz;
  }

  public synchronized void deposit(
                              float money) {
    balance += money;
  }

  public synchronized void withdraw(
                               float money) {
    balance -= money;
  }
}
```

E' possibile chiamare il metodo `synchronized` su qualsiasi oggetto di cui si ha una referenza (quindi escludendo i primitivi):

```
class Account {
  private float balance;
  private final Object lock = new Object();
  ...
  public void deposit(float money) {
    synchronized(lock) {
      balance += money;
    }
  }
  public void withdraw(float money) {
    synchronized(lock) {
      balance -= money;
    }
  }
}
```

Questo si rivela molto utile per evitare di sincronizzare eccessivamente, andando a ridurre il parallelismo tra i thread (cosa che nei TdE solitamente è richiesta). Si veda per esempio l'esercizio svolto sotto, dove si ha un'intera array di locks invece di utilizzare unicamente il singolo lock di `this`;

Altre cose su cui riporre particolare attenzione quando si utilizza un intrinsic lock:

- essendo il lock legato ad un'istanza di una classe, quando si utilizza `synchronized` in metodi statici non si ha a disposizione un'istanza su cui sincronizzare. Si utilizza quindi l'istanza della classe stessa, ovvero:

```
public class ExampleClass {
  public static void static1() {
    synchronized(ExampleClass.class) {
      // sezione critica
    }
  }
  // Di default Java effettua la stessa cosa
  // anche per synchronized posto su un metodo
  // statico. I due metodi sono equivalenti
  public static synchronized void static2() {
    // sezione critica
  }
}
```

- `synchronized` posto su un metodo non sarà automaticamente ereditato da metodi che ne effettuano l'overriding. Bisognerà quindi richiedere esplicitamente che anche il l'overriding method sia synchronized.

Variabili volatili e variabili atomiche

Se non necessitiamo di una sezione critica, ma solo di forzare la sincronizzazione di una variabile della cache locale del thread con la memoria centrale, possiamo utilizzare la keyword `volatile` sulla variabile di interesse:

```
class StudentGrade {
  private final String matricola;
  private volatile int grade;

  public StudentGrade(String matricola) {
    this.matricola = matricola;
  }

  public void getGrade() {
    return grade;
  }

  public void setGrade(int grade) {
    this.grade = grade;
  }
}
```

Lo svantaggio è la mancanza della sezione critica stessa: un'operazione come `x++` non è atomica, e se non sincronizzata causerà problemi a prescindere dalla presenza di volatile. Questo problema viene risolto dall'utilizzo delle variabili atomiche, che sono implementate dalle classi `AtomicBoolean`, `AtomicInteger`, `AtomicLong` e `AtomicReference<V>`.

```
class Counter {
  private final AtomicInteger count =
      new AtomicInteger();

  public int get() {
    return count.get();
  }

  public void increment() {
    count.getAndIncrement();
  }

  public void set(int count) {
    count.getAndSet(count);
  }
}
```

La mancanza di un tipo atomico per double e float può essere sopperita mediante:

- float: AtomicInteger con i metodi `Float.floatToIntBits(float)` e `Float.intBitstoFloat(int)`
- double: AtomicLong con i metodi `Double.doubleToLongBits(double)` e `Double.longBitsToDouble(long)`

oppure, se è necessario sommare, utilizzando `DoubleAdder`:

```
class Account {
  private final DoubleAdder balance =
      new DoubleAdder();

  public float get() {
    return (float) balance.sum();
  }

  public void deposit(float money) {
    balance.add(money);
  }

  public void withdraw(float money) {
    balance.add(-money);
  }
}
```

Particolare attenzione va posta anche nella dichiarazione di array: `volatile int[] array = new int[20]` dichiara un array di interi con referenza volatile, *non* un array di interi volatili. Bisogna quindi utilizzare `AtomicIntegerArray`, `AtomicLongArray` o `AtomicReferenceArray<E>`.

Variabili finali e oggetti immutabili

Le variabili `final` non hanno bisogno di essere sincronizzate in alcuna maniera, in quanto, non potendo cambiare, è impossibile che avvenga un desync tra la cache locale e la memoria centrale.

Questo ci porta agli oggetti immutabili: un oggetto si dice **immutabile** se ogni suo campo è dichiarato `final` e ogni modifica dello stesso avviene mediante la creazione di un nuovo oggetto:

```
public class ImmutableRGB {
  private final String name;
  // Values must be between 0 and 255.
  private final int red;
  private final int green;
  private final int blue;

  private int check(int component) {
    if (component < 0 || component > 255)
      throw new IllegalArgumentException();
    return component;
  }

  public ImmutableRGB(int r, int g, int b,
                      String name) {
    this.red = check(r);
    this.green = check(g);
    this.blue = check(b);
    this.name = name;
  }

  public int getRGB() {
    return (red << 16) | (green << 8) | blue;
  }

  public String getName() {
    return name;
  }

  public ImmutableRGB invert() {
    return new ImmutableRGB(255 - red,
            255 - green,
            255 - blue,
            "Inverse of " + name);
  }
}
```

Gli oggetti immutabili non necessitano di alcun tipo di sincronizzazione.

Variabili `static`

Al contrario delle variabili `final`, le variabili `static` non sono necessariamente thread-safe, in quanto possono essere lette e modificate da più classi e più thread contemporaneamente. Bisognerebbe quindi sempre sincronizzare l'accesso alle variabili `static` per evitare che si leggano valori non aggiornati.

`wait`, `notify` e `notifyAll` (producer/consumer pattern)

Java mette a disposizione nell'intrinsic lock anche delle primitive per implementare dei **Guarded Blocks**, ovvero delle condizioni che devono essere vere prima che l'esecuzione di un metodo possa continuare:

- `wait() throws InterruptedException`: sospende l'esecuzione di un thread fino a quando viene risvegliato da un altro, rilasciando il lock al momento della sospensione e ri-acquisendolo al risveglio
- `notify()`: sveglia un thread sospeso in attesa sul lock di questo oggetto per riprendere la sua esecuzione
- `notifyAll()`: sveglia tutti i thread sospesi in attesa sul lock di questo oggetto

L'invocazione di questi metodi richiede che il thread invocante sia possessore del lock stesso.

Notiamo inoltre come il metodo wait sollevi l'eccezione `InterruptedException`, che avviene se il thread corrente viene interrotto (ad esempio perché viene richiesta la terminazione del programma).

Riprendendo l'esempio di un account bancario, se volessimo aspettare che l'utente abbia soldi prima di effettuare un'operazione di prelievo:

```
class Account {
  private final Object lock = new Object();
  private float balance;
  public Account(float b){ this.balance = b; }

  public void deposit(float money) {
    synchronized(lock) {
      balance += money;
      lock.notifyAll();
    }
  }
  public void withdraw(float money)
        throws InterruptedException {
    synchronized(lock) {
      // Notiamo l'utilizzo del while al posto
      // di un if: questo perché il thread può
      // svegliarsi spontaneamente (su alcune
      // architetture e/o OS specifici)
      while (balance - money < 0)
        lock.wait();
      balance -= money;
    }
  }
}
```

Deadlock

Quando si ha a che fare con due o più lock, bisogna prestare particolare attenzione a non generare deadlock. Di seguito sono riportati i casi più comuni in cui questo si può verificare.

- Deadlock su due lock della stessa istanza della classe (acquisizione di lock alternati):

```
public class SyncObject {
  private final Object xLock = new Object();
  private final Object yLock = new Object();
  ...
  public void method1() {
    synchronized(xLock) {
      synchronized(yLock) {
        ...
      }
    }
  }
  public void method2() {
    synchronized(yLock) {
      synchronized(xLock) {
        ...
      }
    }
  }
}
```

Se un thread ha acquisito xLock in method1 e un altro ha acquisito yLock in method2, entrambi devono aspettare l'altro thread per poter acquisire il lock che gli manca. Bisogna sempre acquisire i lock nel medesimo ordine.

- Deadlock su lock di due istanze diverse della classe:

```
public class Vector {
  ...
  public synchronized void sum(Vector other) {
    synchronized(other) {
      this.x += other.x;
      this.y += other.y;
    }
  }
}
```

Dati v1, v2, se un thread chiama v1.sum(v2) e allo stesso tempo un altro chiama v2.sum(v1), si ha ancora una volta un'acquisizione di lock alternata: potenzialmente un deadlock. Bisogna acquisirne uno alla volta:

```
public class Vector {
  ...
  public void sum(Vector other) {
        double otherX, otherY;
      synchronized(other) {
        otherX = other.x;
        otherY = other.y;
      }

      synchronized(this) {
        this.x += otherX;
        this.y += otherY;
    }
  }
}
```

- Deadlock con 2 lock e `wait()`:

```
public class Bank {
  ...
  public double getBalance(String clientName) {
    synchronized(accNames) {
      synchronized(accBalances) {
        int pos;
        while((pos = accNames.indexOf(clientName)) == -1)
          accNames.wait();
        return accBalances.get(pos);
      }
    }
  }

  public void addAccount(String clientName) {
    synchronized(accNames) {
      synchronized(accBalances) {
        accNames.add(clientNames);
        accBalances.add(0.0);
        accNames.notifyAll();
      }
    }
  }
}
```

Quando `getBalance` chiama `accNames.wait()`, rilascia il lock su accNames, ma non quello su accBalances, quindi `addAccount` non potrà mai arrivare alla corrispondente `notifyAll()`. Bisogna quindi chiamarlo quando abbiamo acquisito solo il primo lock:

```
public class Bank {
  ...
  public double getBalance(String clientName) {
    synchronized(accNames) {
      int pos;
      while((pos = accNames.indexOf(clientName)) == -1)
        accNames.wait();

      synchronized(accBalances) {
        return accBalances.get(pos);
      }
    }
  }
  ...
}
```

Explicit lock

Meccanismi di lock più sofisticati sono forniti dal package `java.util.concurrent.locks`. Questo definisce un'interfaccia di base `Lock` che, oltre a permettere le stesse operazioni dell'intrinsic lock, aggiunge anche ulteriori metodi e funzionalità:

- Permette di chiamare `lock` e `unlock` anche in metodi separati
- Il lock è sempre esplicito, bisogna per forza pensare a chi ne avrà accesso
- `lockInterruptibly() throws InterruptedException`: esegue la stessa operazione di lock, ma permette di interrompere l'esecuzione del thread quando questo è stato sospeso.
- `boolean tryLock()`: acquisisce il lock solo se non è già utilizzato

```
Lock lock = new ReentrantLock();
if (lock.tryLock()) {
  try {
    // qui siamo in sezione critica
  } finally {
    // 'finally' per rilasciare il lock qualsiasi cosa succeda
    // sia esecuzione normale che eccezioni
    lock.unlock();
  }
} else {
  // azioni alternative dove il lock non è necessario
}
```

Executors

L'utilizzo eccessivo/non limitato di thread può portare a problemi di performance. Per ovviare a questo problema, sono state introdotte le interfacce `Executor`, `ExecutorService` e `ScheduledExecutorService` e i metodi statici nella classe `Executors` per istanziarne implementazioni concrete. Per esempio `Executors.newFixedThreadPool(int)` utilizza un numero fisso di thread a cui fare eseguire le task.

```
ExecutorService executor = Executors.newFixedThreadPool(5);
executor.submit(() -> {
  // Questo viene eseguito su uno dei 5 thread
  System.out.println("Executed on " +
      Thread.currentThread());
});
```

Collezioni concorrenti

Sempre il package `java.util.concurrent` introduce anche dei tipi di collezioni che sono già predisposte all'utilizzo concorrente, senza necessitare di sincronizzazione. Di seguito una per tipo di collezione:

- `CopyOnWriteArrayList<E>` implementa `List<E>`
- `ConcurrentHashMap<K, V>` implementa `ConcurrentMap<K, V>` che implementa `Map<K, V>`
- `ConcurrentHashMap.<K>newKeySet()` implementa `Set<K>`
- `ConcurrentLinkedQueue` implementa `Queue<E>` (in maniera non bloccante)
- `ArrayBlockingQueue` implementa `BlockingQueue<E>` che implementa `Queue<E>` (aggiungendo operazioni bloccanti)

Esercizio (TdE del 2022-07-21, esercizio 2 - punto a e b)

Testo dell'esercizio

Si consideri la seguente classe `ScoreBoard` per memorizzare i voti di un insieme di studenti passato in fase di costruzione:

```
public class ScoreBoard {
  private String[] students;
  private int[] scores;

  public ScoreBoard(String[] stud) {
    students = new String[stud.length];
    for(int i = 0; i < stud.length; i++)
      students[i] = stud[i];
    scores = new int[stud.length];
  }

  public int getScore() {
    for(int i = 0; i < students.length; i++) {
      if(students[i].equals(stud))
        return scores[i];
    }
    return -1;
  }

  public int setScore(String stud, int score) {
    for(int i = 0; i < students.length; i++) {
      if(students[i].equals(stud))
        scores[i] = score;
    }
  }
}
```

Domanda a)

Si introduca l'opportuno codice di sincronizzazione nei metodi `getScore` e `setScore` al fine di massimizzare il parallelismo, consentendo a thread diversi di accedere in parallelo ai dati di studenti diversi, evitando conflitti nell'accesso ai dati (voto) del medesimo studente.

Soluzione

Osserviamo per prima cosa come, non potendo cambiare né il costruttore né la definizione dei campi, possiamo scartare a priori variabili volatili e atomiche. Siamo quindi obbligati ad utilizzare intrinsic locking (strano).

Come seconda osservazione, notiamo come students sia inizializzata nel costruttore e non sia poi più modificata. Questo vuol dire che, molto probabilmente, l'accesso a questa non dovrà stare in sezione critica.

Come ultima cosa, per massimizzare il parallelismo, dobbiamo evitare il conflitto dei dati tra medesimi studenti, non tra studenti diversi: non dobbiamo quindi utilizzare lo stesso lock per ogni studente, dovremmo avere un lock per studente. Non potendo dichiarare nessun campo aggiuntivo, possiamo riutilizzare come intrinsic lock i valori dentro all'array students, in quanto come detto in precedenza non vengono modificati.

```
public class ScoreBoard {

  ...

  public int getScore() {
    for(int i = 0; i < students.length; i++) {
      if(students[i].equals(stud)) {
        synchronized(students[i]) {
          return scores[i];
        }
      }
    }
    return -1;
  }

  public int setScore(String stud, int score) {
    for(int i = 0; i < students.length; i++) {
      if(students[i].equals(stud)) {
        synchronized(students[i]) {
          scores[i] = score;
        }
      }
    }
  }
}
```

Domanda b)

Si modifichi il metodo `getScore` affinché sospenda il chiamante se il voto dello studente cercato è minore o uguale a zero. Si specifichi se e come sia necessario modificare altri metodi.

Soluzione

Questo è un esempio di consumer/producer standard:

- La condizione di wait sarà scores[i] <= 0
- Bisogna modificare `setScore` per notificare l'avvenuta modifica

```
public class ScoreBoard {

  ...

  public int getScore() {
    for(int i = 0; i < students.length; i++) {
      if(students[i].equals(stud)) {
        synchronized(students[i]) {
          while(scores[i] <= 0)
            students[i].wait();
          return scores[i];
        }
      }
    }
    return -1;
  }

  public int setScore(String stud, int score) {
    for(int i = 0; i < students.length; i++) {
      if(students[i].equals(stud)) {
        synchronized(students[i]) {
            scores[i] = score;
            students[i].notifyAll();
        }
      }
    }
  }
}
```

Programmazione funzionale

Il paradigma di programmazione funzionale si basa sul principio di considerare le funzioni come oggetti, e quindi utilizzarli come tali. In questo modo, le funzioni possono essere passate come parametri, restituite come risultati, memorizzate in variabili, ecc. Le funzioni usate come parametro senza specificare il nome della funzione, ma solo il comportamento che essa deve avere, sono dette **funzioni anonime**.

Espressioni lambda

Supponiamo di avere una classe fatta in questo modo:

```
class Persona {
  private String nome;
  private int eta;

  public Persona(String nome, int eta) {
    this.nome = nome;
    this.eta = eta;
  }

  public String getNome() {
    return nome;
  }

  public int getEta() {
    return eta;
  }
}
```

e supponiamo di avere una lista di persone:

```
List<Persona> persone = new ArrayList<>();
persone.add(new Persona("Mario", 20));
persone.add(new Persona("Luigi", 30));
persone.add(new Persona("Pippo", 40));
persone.add(new Persona("Pluto", 50));
```

Possiamo, ad esempio, ordinare la lista di persone per età, usando il metodo `sort` della classe `Collections`:

```
public static <T> void sort(
  List<T> list, Comparator<? super T> c
)
```

Il metodo `sort` prende come primo parametro una lista di oggetti di tipo `T`, e come secondo parametro un oggetto di tipo `Comparator`, possiamo passare un `Comparator` direttamente usando una **espressione lambda**:

```
obj.method(
  (param1, param2) -> {
    // corpo della funzione
  }
);
```

Nel nostro esempio il metodo `sort` sarà implementato così:

```
Collections.sort(persone, (p1, p2) -> {
  /* Java conosce già il tipo di p1 e p2,
  quindi non è necessario specificarlo */
  if(p1.getEta() > p2.getEta()) {
    return 1; }
  else if(p1.getEta() < p2.getEta()) {
    return -1; }
  else return 0;
});
```

Interfacce funzionali

Nell'esempio di prima, nel metodo `public static <T> void sort(List<T> list, Comparator<? super T> c)`, il secondo parametro è un oggetto di tipo `Comparator<T>`, è una **interfaccia funzionale**.

Una **interfaccia funzionale** è un'interfaccia che prevede di implementare un solo metodo. In questo caso, l'interfaccia `Comparator<T>` prevede di implementare il metodo `int compare(T o1, T o2)`, che impone un ordine tra due oggetti di tipo `T` e restituisce 1, 0, -1 a seconda che il primo parametro o1 sia rispettivamente maggiore, uguale o minore del secondo.

Avendo l'interfaccia funzionale un solo metodo, possiamo usare direttamente una **espressione lambda** per implementarla:

```
Comparator<Persona> comparator = (p1, p2) -> {
  if(p1.getEta() > p2.getEta()) {
    return 1; }
  else if(p1.getEta() < p2.getEta()) {
    return -1; }
  else return 0;
};
```

è come se avessimo *"assegnato una funzione ad una variabile (comparator)"*. Possiamo quindi passare la variabile `comparator` come secondo parametro al metodo `sort`: `Collections.sort(persone, comparator);`. Oppure, possiamo passare direttamente una lambda (come mostrato nel capitolo precedente).

Java fornisce delle `<interfacce funzionali / il corrispondente metodo per eseguire la funzione>` nel package `java.util.function` quali:

- **`Function<T, R>` / `.apply(T t)`**: funzione che prende un parametro di tipo `T` e restituisce un oggetto di tipo `R`;
- **`Consumer<T>` / `.accept(T t)`**: funzione che prende un parametro di tipo `T` e non restituisce nulla;
- **`Supplier<T>` / `.get()`**: funzione che non prende parametri e restituisce un oggetto di tipo `T`;
- **`Predicate<T>` / `.test(T t)`**: funzione che prende un parametro di tipo `T` e restituisce un booleano (true o false);
- **`BiFunction<T, U, R>` / `.apply(T t, U u)`**: funzione che prende due parametri di tipo `T` e `U` e restituisce un oggetto di tipo `R`.

E le corrispondenti per i tipi primitivi `int`, `double` e `long`:

- **`IntFunction<R>` / `.apply(int v)`**: funzione che prende un parametro di tipo `int` e restituisce un oggetto di tipo `R`;
- **`IntConsumer` / `.accept(int v)`**: funzione che prende un parametro di tipo `int` e non restituisce nulla;

- **`IntSupplier` / `.getAsInt()`**: funzione che non prende parametri e restituisce un oggetto di tipo `int`;
- **`IntPredicate` / `.test(int v)`**: funzione che prende un parametro di tipo `int` e restituisce un booleano (true o false);
- **`ToIntFunction<T>` / `.applyAsInt(T v)`**: funzione che prende un parametro di tipo `T` e restituisce un primitivo di tipo `int`: `int applyAsInt(T value)`;
- **`DoubleToIntFunction` / `.applyAsInt(double v)`**: funzione che prende un parametro di tipo `double` e restituisce un oggetto di tipo `int`...

Composizione di funzioni (`Stream<T>`)

Possiamo concatenare delle funzioni che agiscono su una Collezione per costruire una **catena di funzioni**, che filtrerà, mapperà e agirà sulla nostra collezione.

Questa catena inizierà invocando il metodo **`stream()`** sulla collezione. Nell'esempio:

```
Stream<Persona> stream = persone.stream();
```

`filter`

Possiamo filtrare lo `stream` per far si che tutti gli elementi della collezione soddisfino un certo `Predicate<T>`. Per farlo, usiamo il metodo **`Stream<T> filter(<predicato>)`**. Nel nostro esempio, vogliamo filtrare le persone che hanno più di 30 anni:

```
Stream<Persona> stream = persone.stream()
  .filter(p -> p.getEta() >= 30);
```

La funzione lambda nel metodo `filter` sarà `true` per tre persone: Luigi, Pippo e Pluto. Verrà quindi eliminato Mario (che ha 20 anni) dal nostro `stream`.

`map`

`Stream<U> map(<funzione da T a U>)` prende in input una funzione (`Function<T, U>`) e la applica ad ogni elemento dello `stream`. Si usa per trasformare gli elementi dello `stream` in altri elementi. Nel nostro esempio, vogliamo trasformare le persone (già filtrate) in stringhe contenenti il nome e l'età:

```
Stream<String> stream = persone.stream()
  .filter(p -> p.getEta() >= 30)
  .map(p -> {
    return p.getNome() + ": " +
    p.getEta().toString();
  });
```

Otterrò uno `stream`, contenente le stringhe: "Luigi: 30", "Pippo: 40", "Pluto: 50".

Esistono anche metodi simili per mappare a valori primitivi:

- **`IntStream mapToInt(<funzione da T a int>)`**,
- **`DoubleStream mapToDouble(<funzione da T a double>)`**
- **`LongStream mapToLong(<funzione da T a long>)`**.

Questi ritornano un'istanza di uno stream di valori primitivi, permettendo di chiamare metodi aggiuntivi che possono essere utili (vedi sotto).

`distinct` e `sorted`

- **`Stream<T> distinct()`** elimina gli elementi duplicati dello `stream`.

- **`Stream<T> sorted()`** ordina gli elementi dello `stream` secondo il loro ordinamento naturale. Funziona solo su tipi che implementano già l'interfaccia `Comparable` (come ad esempio `Integer` e `String`).

- **`Stream<T> sorted(Comparator<? super T> c)`** ordina gli elementi dello `stream` in base a quanto definito dal `Comparator` dato.

Per facilitare la creazione di `Comparator` esistono anche una serie di metodi, che permettono di mappare oggetti a dei loro campi per il quale un ordine naturale è già definito (come ad esempio per stringhe e primitivi):

- **`static <T,U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T,? extends U> keyExtractor)`**: accetta una funzione che estrae un campo dall'oggetto dato e costruisce un comparatore che compara su tale campo. Per esempio, il comparatore definito precedentemente (che utilizza l'età delle persone), può essere scritto come:

```
Comparator<Persona> comparator =
  Comparator.comparing(p -> p.getEta())
```

- **`Comparator<T> reversed()`**: Restituisce un comparatore che impone l'ordine inverso di quello corrente. Per esempio, per ordinare una lista di persone per età in ordine decrescente:

```
List<Persona> persone = new ArrayList<>();
persone.add(new Persona("Mario", 20));
persone.add(new Persona("Luigi", 30));
persone.add(new Persona("Pippo", 40));
persone.add(new Persona("Pluto", 50));
Collections.sort(persone, Comparator
  .comparing(p -> p.getEta())
  .reversed());
// persone sarà [Pluto, Pippo, Luigi, Mario]
```

- **`static <T extends Comparable<? super T>> Comparator<T> naturalOrder()`** e
- **`static <T extends Comparable<? super T>> Comparator<T> reverseOrder()`**: restituiscono un comparatore di un oggetto T per il quale è già definito un proprio ordine naturale che compara in ordine naturale/inverso. Per esempio, per ordinare una lista di interi in ordine decrescente: (*l'esempio è nella facciata seguente*)

```
List<Integer> a = new ArrayList<>();
a.add(20);
a.add(30);
a.add(40);
a.add(50);
Collections.sort(a, Comparator.reverseOrder())
// a sarà [50, 40, 30, 20]
```

flatMap

`Stream<U> flatMap(<funzione da T a Stream<U>>)` prende in input una funzione (`Function<T, Stream<U>>`), la applica ad ogni elemento di tipo T dello `stream`. La funzione restituisce un altro `stream` (in generale contenente elementi di tipo diverso U) per ogni elemento. Infine tutti gli `stream` restituiti dalla funzione vengono concatenati in un unico `stream`.

Nel nostro esempio, vogliamo trasformare ogni persona (sempre filtrata con età >= 30) in una lista di stringhe contenenti il nome e l'età, e poi concatenare tutti gli `stream` in un unico `stream`:

```
Stream<String> stream = persone.stream()
  .filter(p -> p.getEta() >= 30)
  .flatMap(p -> {
    List<String> listaPerP;
    listaPerP = new ArrayList<>();
    listaPerP.add(p.getNome());
    listaPerP.add(p.getEta().toString());
    return listaPerP.stream();
  });
```

Otterrò uno `stream`, contenente le stringhe: "Luigi", "30", "Pippo", "40", "Pluto", "50".

Esistono anche metodi simili per flat-mappare a valori primitivi:

- **`IntStream flatMapToInt(<funzione da T a IntStream)`**,
- **`DoubleStream flatMapToDouble(<funzione da T a DoubleStream)`**
- **`LongStream flatMapToLong(<funzione da T a LongStream)`**.

Questi ritornano un'istanza di uno stream di valori primitivi, permettendo di chiamare metodi aggiuntivi che possono essere utili (vedi sotto).

reduce

La **`T reduce(Identità, <funzione binaria>)`** è un'operazione di aggregazione che restituisce un singolo valore a partire da uno `stream`.

Si ha come primo parametro un valore iniziale (identità), e come secondo parametro una funzione binaria che prende in input due elementi dello `stream` e restituisce un altro elemento (dello stesso tipo) ottenuto a partire dai due. La funzione verrà applicata induttivamente ad ogni elemento dello `stream`, fino ad ottenere un solo elemento.

Nel nostro esempio, vogliamo ottenere la somma delle età delle persone (filtrate):

```
Integer sommaEta = persone.stream()
  .filter(p -> p.getEta() >= 30)
  .mapToInt(p -> p.getEta())
  .reduce(
    0,
    (eta1, eta2) -> eta1 + eta2 );
```

Otterrò un intero: 120. Nelle varie iterazioni nello `stream` contenente le età `[30, 40, 50]`, la `reduce` si applica in questo modo:

- 0 (valore iniziale) + 30 (primo elemento dello stream) = 30 (risultato parziale),
- 30 (risultato parziale della iterazione precedente) + 40 (secondo elemento dello stream) = 70,
- 70 + 50 = 120.

Esiste in aggiunta anche un overload che non richiede un'identità **`Optional<T> reduce(<funzione binaria>)`**. Questo permette di effettuare la medesima operazione di riduzione, sapendo però in aggiunta se lo stream fosse vuoto o contenesse degli elementi:

- Una riduzione come la precedente su `[30, 40, 50]` restituisce ancora 120
- Una riduzione come la precedente su `[0]` restituisce ancora 0
- Una riduzione come la precedente su `[]` restituisce `Optional.empty()` invece di 0

collect

Siamo ora interessati ad avere una Collezione (List, Set, ...) invece che uno stream. Per fare ciò, possiamo usare il metodo **`Collection<T> collect(<funzione che restituisce una Collettore>)`**. Nel nostro esempio, vogliamo ottenere una lista di stringhe contenente nome: età delle persone (filtrate con età >= 30):

```
List<String> lista = persone.stream()
  .filter(p -> p.getEta() >= 30)
  .map(p ->
    p.getNome() + ": " +
    p.getEta().toString()
  );
  .collect(Collectors.toList());
```

Esistono diversi tipi di collettori già definiti:

- **`toList()`** e **`toSet()`** per avere una lista o un set
- **`toCollection(Supplier<C> collectionFactory)`** per avere un altro tipo di `Collection` o per avere una specifica implementazione di `List` o `Set`
- **`joining()`**
- **`joining(CharSequence delimiter)`**
- **`joining(CharSequence delimiter, CharSequence prefix, CharSequence suffix)`** per concatenare un insieme di stringhe in una stringa unica

`min` e `max`

`Optional<T> max(Comparator<? super T> c)` e **`Optional<T> min(Comparator<? super T> c)`** restituiscono rispettivamente l'oggetto massimo o minimo all'interno dello stream (se presente), secondo l'ordine definito dal `Comparator` dato.

```
// Restituisce la persona più anziana, Pluto
Optional<Persona> piuAnziano = Arrays.asList(
  new Persona("Mario", 20),
  new Persona("Luigi", 30),
  new Persona("Pippo", 40),
  new Persona("Pluto", 50))
  .stream()
  .max(Comparator.comparing(p -> p.getEta()));
```

`anyMatch`, `allMatch` e `noneMatch`

`boolean anyMatch(<predicato>)`, **`Optional<T> allMatch(<predicato>)`** e **`Optional<T> noneMatch(<predicato>)`** restituiscono rispettivamente se almeno uno, tutti o nessuno degli elementi dello stream rispetta il predicato;

```
List<Persona> persone = Arrays.asList(
  new Persona("Mario", 20),
  new Persona("Luigi", 30),
  new Persona("Pippo", 40),
  new Persona("Pluto", 50));
// Restituisce vero,
// Pippo e Pluto sono abbastanza anziani
boolean any = persone.stream()
  .anyMatch(p -> p.getEta() > 35);
// Restituisce falso,
// Mario e Luigi sono più giovani
boolean all = persone.stream()
  .allMatch(p -> p.getEta() > 35);
// Restituisce falso,
// Pippo e Pluto sono più anziani
boolean all = persone.stream()
  .noneMatch(p -> p.getEta() > 35);
```

`forEach`

Possiamo usare il metodo **`forEach(<funzione da T a void>)`** per eseguire una funzione per ogni elemento dello stream. Nel nostro esempio, vogliamo stampare il nome e l'età di ogni persona (filtrata con età >= 30):

```
persone.stream()
  .filter(p -> p.getEta() >= 30)
  .forEach(p ->
    System.out.println(p.getNome() + ": "
    + p.getEta().toString()));
```

`IntStream`, `DoubleStream` e `LongStream`

Esistono delle specializzazioni della classe Stream per i valori primitivi, ottenibili mediante opportune chiamate a un normale stream con `mapToInt`, `flatMapToInt`, etc.

Queste interfacce definiscono metodi aggiuntivi per facilitare le operazioni di aggregazione, quali:

- **`OptionalDouble average()`**
- **`OptionalInt max()`**
- **`OptionalInt min()`**
- **`int sum()`**

Per convenienza questi sono scritti solo per IntStream, ma sono uguali per gli altri, basta semplicemente sostituire `int` con `double`/`long` e `OptionalInt` con `OptionalDouble`/`OptionalLong`

`Optional<T>`, `OptionalInt`, `OptionalDouble` e `OptionalLong`

L'oggetto **`Optional`** è un contenitore per un valore che può essere nullo. È usato per evitare di avere eccezioni di tipo `NullPointerException`.

Esistono anche le specializzazioni per tipi primitivi `OptionalInt`, `OptionalDouble` e `OptionalLong`. Il loro utilizzo è pressoché identico alla normale classe `Optional`.

Possiamo creare un `Optional` con il valore `val` con il metodo `Optional.of(val)`. Oppure con il metodo `Optional.empty()` per creare un `Optional` vuoto (sostituisce il null).

Sono poi disponibili i metodi per lavorare con gli Optional:

- **`ifPresent(<funzione>)`** che applica la funzione in input solo se l'Optional non è vuoto.
- **`orElse(<val>)`** che restituisce il valore dell'Optional se non è vuoto, altrimenti restituisce il valore `val` in input.
- **`Optional<U> flatMap(<funzione da T a Optional<U>)`** restituisce un `Optional<U>`, applicando la funzione in input solo se l'`Optional` non è vuoto; altrimenti ritorna un `Optional` vuoto.

Facciamo un esempio usando gli `Optional` e i tre metodi visti in precedenza:

```
Optional<String> opt = Optional.of("ciao");

// Stampa "ciao"
opt.ifPresent(s -> System.out.println(s));

// Stampa "ciao" perché opt non è vuoto
System.out.println(opt.orElse("vuoto"));

/* La funzione in flatMap viene applicata
e restituisce un Optional<String>
contenente "ciao mondo",
quindi viene stampato "ciao mondo"
per effetto della orElse */
System.out.println(
  opt .flatMap(
    s -> Optional.of(s + " mondo"))
  .orElse("vuoto"));


Optional<String> opt2 = Optional.empty();

// Non stampa nulla perché opt2 è vuoto
opt2.ifPresent(s -> System.out.println(s));

// Stampa "vuoto" perché opt2 è vuoto
System.out.println(opt2.orElse("vuoto"));

/* La funzione in flatMap non viene applicata
e restituisce un Optional<String> vuoto,
quindi viene stampato "vuoto"
per effetto della orElse */
System.out.println(
  opt2.flatMap(
    s -> Optional.of(s + " mondo"))
  .orElse("vuoto"));
```

Otterremo in output:

```
  ciao
  ciao
  ciao mondo
  vuoto
  vuoto
```

Closures

Viene detta closure la combinazione di una funzione insieme a una referenza allo stato che la circonda.

Detto in maniera diversa, data una funzione lambda, si dice che questa "si chiuda attorno" (ovvero catturi) le variabili all'interno del suo scope di definizione, permettendo di utilizzarle all'interno della lambda stessa.

Con un esempio:

```
final String prefix = "Z";
Predicate<String> pred = s ->
    s.startsWith(prefix);
```

la lambda `pred` cattura la variabile prefix, utilizzandola al suo interno, prendendo quindi il nome di closure.

Per evitare effetti collaterali (*side effects*), viene consentito di catturare solo variabili `final` oppure **effectively final**, ovvero non dichiarata final ma usata come se fosse tale.

```
String prefix = "Z";
// Altre istruzioni che fanno cose
String nonFinalPrefix = prefix + "aaaa";
System.out.println(nonFinalPrefix);
nonFinalPrefix = "";
// prefix non è dichiarata final, ma non è
// mai modificata: è effectively final
Predicate<String> pred = s ->
    s.startsWith(prefix);
// Errore di compilazione: nonFinalPrefix é
// modificata, quindi non è effectively final
Predicate<String> pred2 = s ->
    s.startsWith(nonFinalPrefix);
```

Esercizio sulla programmazione funzionale (TdE del 2019-02-18, esercizio 4 - punto c)

Testo dell'esercizio

Si consideri il seguente metodo statico, che ha la precondizione che ciascun elemento di nums è una stringa corrispondente a una rappresentazione testuale di un numero intero

```
public static List<Integer> addX
(List<String> nums, int x) {
  List<Integer> plusX = new LinkedList<>();
  for(String numString : nums) {
    number = Integer.valueOf(numString);
    if (number>0)
      plusX.add(number + x);
  }
  return plusX;
}
```

Si riscriva il metodo utilizzando i costrutti della programmazione funzionale di Java 8.

Soluzione

Il metodo `addX` non fa altro che prendere in input una lista di stringhe `nums` e un intero `x`, e restituire una lista di interi ottenuta sommando `x` a tutti gli interi positivi presenti in `nums` (gli interi <= 0 nella lista `nums` non vengono aggiunti nella lista `plusX`).

Quindi possiamo:

- applicare il metodo `stream()` alla lista di stringhe per iniziare la nostra "catena di funzioni",
- convertire ogni stringa della lista di partenza in un intero (ritornando poi lo `stream` di interi), tramite una `map` con parametro parametro il metodo `Integer.valueOf(numString)`,
- filtrare gli interi positivi,
- sommare `x` a ciascuno di essi.

```
public static List<Integer> addX
(List<String> nums, int x) {
  return nums.stream()
    .map(numString ->
      Integer.valueOf(numString))
    .filter(number -> number > 0)
    .map(number -> number + x)
    .collect(Collectors.toList());
}
```

Design pattern

I design pattern sono dei modelli di soluzione a problemi comuni di progettazione software. Esistono diversi tipi di design pattern, ma in generale si possono suddividere in 4 categorie: *creazionali, strutturali, comportamentali e architetturali*.

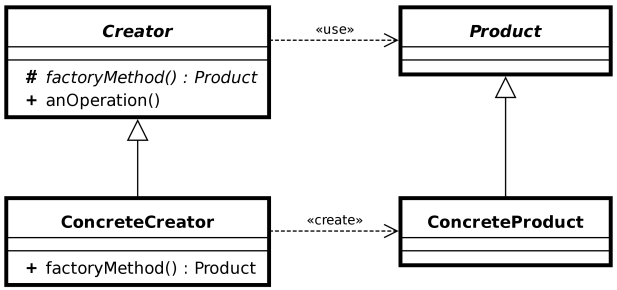
Pattern creazionali

I design pattern creazionali sono quelli che si occupano di risolvere problemi relativi alla *creazione di oggetti*.

Factory Method

Il **Factory Method** permette di creare oggetti senza specificare la loro classe concreta. Questo pattern è particolarmente utile quando abbiamo bisogno di istanziare uno o più oggetti astratti per poi effettuarvi delle operazioni che non dipendono dal loro tipo concreto (uguale per tutti gli oggetti). Come è ben noto non è possibile costruire un oggetto astratto. Per ovviare a questo problema si usa il **Factory Method** che permette al Client di specificare al metodo che dovrà effettuare tali operazioni, solo la *factory* con cui costruire gli oggetti astratti, il metodo poi potrà lavorare con gli oggetti astratti e non sarà necessario creare un metodo diverso (che effettua le stesse operazioni) per ciascun tipo concreto, solo per avere accesso al costruttore del tipo concreto in questione.

UML



Nell'UML, la classe astratta `Creator`, che ha bisogno di creare un oggetto `Product`, delega la creazione ad una classe `ConcreteCreator` (che estende `Creator`). La classe `ConcreteCreator` implementa il metodo `factoryMethod()` che crea un oggetto `ConcreteProduct` (che estende `Product`).

Esempio

Supponiamo di voler creare un programma per creare dei documenti. Il programma deve essere in grado di creare documenti di diversi tipi (documenti di testo, di calcolo, pdf, ...) decisi a runtime dall'utente. Il programma non può prevedere che tipo di documento l'utente deciderà di creare, quindi il **Factory Method** è la soluzione migliore.

```
public abstract class Document {
  private String title;

  public abstract void
    setTitle(String title);
  ...
}

public class TextDocument extends Document {
  @Override
  public void setTitle(String title) {
    this.title = title + ".txt";
  }
  ...
}

public class PDFDocument extends Document {
  @Override
  public void setTitle(String title) {
    this.title = title + ".pdf";
  }
  ...
}

public abstract class DocumentFactory {
  public abstract Document createDocument();
}

public class TextDocumentFactory
extends DocumentFactory {
  @Override
  public Document createDocument() {
    return new TextDocument();
  }
}

public class PDFDocumentFactory
extends DocumentFactory {
  @Override
  public Document createDocument() {
    return new PDFDocument();
  }
}

public class Main {
  /* Crea n documenti dello stesso tipo
  e ne setta il titolo */
  private Document[] createNDocuments
  (int n, DocumentFactory factory) {
    Document[] documents=new Documents[n];
    for (int i = 0; i < n; i++) {
      documents[i]
        = factory.createDocument();
        documents[i]
          .setTitle(String.valueOf(i));
      }

      return documents;
  }

  public static void main(String[] args) {
    Document[] docs;
    DocumentFactory factory;

    // Creazione di n documenti di testo
    factory = new TextDocumentFactory();
    docs = createNDocuments(10, factory);

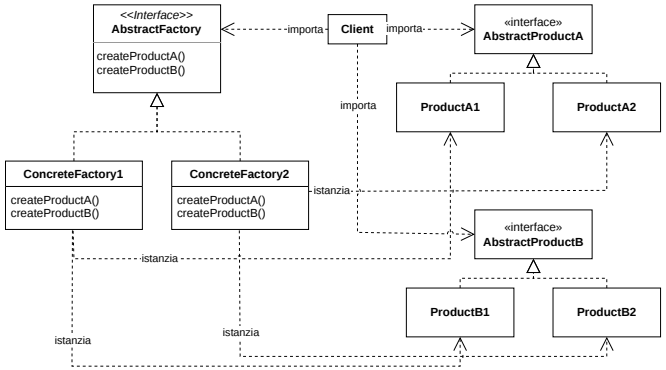
    // Creazione di n documenti PDF
    factory = new PDFDocumentFactory();
    docs = createNDocuments(5, factory);
  }
}
```

In questo modo, il metodo nel `Main` crea n documenti allo stesso modo, senza dover conoscere il tipo di documento da creare, grazie al **Factory Method** che richiama il metodo `createDocument()`. Sarà il Client a decidere quale *factory* inserire nel metodo `createNDocuments()`. Senza il **Factory Method**, ci sarebbero stati diversi metodi `createNDocuments()` per ogni tipo di documento.

Abstract Factory

A differenza del Factory Method, l'**Abstract Factory** permette di creare *famiglie* di oggetti correlati senza specificare la loro classe concreta.

UML



In questo diagramma, si hanno 2 famiglie di oggetti: `AbstractProductA` e `AbstractProductB`, e i rispettivi prodotti concreti `ProductA1`, `ProductA2`, `ProductB1` e `ProductB2`. La classe astratta `AbstractFactory` ha bisogno di creare una famiglia di oggetti, e delega la creazione a 2 classi concrete `ConcreteFactory1` e `ConcreteFactory2`. Le classi concrete implementano i metodi `createProductA()` e `createProductB()` che creano rispettivamente un oggetto `ProductA1` o `ProductA2`, e un oggetto `ProductB1` o `ProductB2`. Il Client, che ha bisogno di creare una famiglia di oggetti, crea una classe `ConcreteFactory` e usa i metodi `createProductA()` e `createProductB()` per creare la famiglia di oggetti.

Esempio

Supponiamo di implementare un insieme di classi dedicate alla creazione di interfacce grafiche. I diversi elementi grafichi possono essere creati secondo stili diversi (PM e Motif). Per ogni elemento grafico viene creata una classe astratta, per ogni stile ed elemento grafico viene creata una classe concreta. Il Client deve essere in grado di modificare lo stile dell'interfaccia grafica a runtime, senza dover modificare il codice.

```
public interface Button { /* ... */ }

public interface ScrollBar { /* ... */ }

public interface GUIFactory {
  public Button createButton();
  public ScrollBar createScrollBar();
}

public class PMButton
  implements Button { /* ... */ }

public class PMScrollBar
  implements ScrollBar { /* ... */ }

public class PMGUIFactory
  implements GUIFactory {
  @Override
  public Button createButton() {
    return new PMButton();
  }

  @Override
  public ScrollBar createScrollBar() {
    return new PMScrollBar();
  }
}

public class MotifButton
  implements Button { /* ... */ }

public class MotifScrollBar
  implements ScrollBar { /* ... */ }

public class MotifGUIFactory
  implements GUIFactory {
  @Override
  public Button createButton() {
    return new MotifButton();
  }

  @Override
  public ScrollBar createScrollBar() {
    return new MotifScrollBar();
  }
}

public class Main {
  public static void main(String[] args) {
    GUIFactory factory;
    Button button;
    ScrollBar scrollBar;

    // Creazione interfaccia grafica PM
    factory = new PMGUIFactory();
    button = factory.createButton();
    scrollBar = factory.createScrollBar();

    // Creazione interfaccia grafica Motif
    factory = new MotifGUIFactory();
    button = factory.createButton();
    scrollBar = factory.createScrollBar();
  }
}
```

In questo modo, il `Main` crea solo la classe `ConcreteFactory` della famiglia richiesta dall'utente e richiama i metodi generici `createButton()` e `createScrollBar()` per creare la famiglia di oggetti, ma sarà la `ConcreteFactory` a preoccuparsi di creare gli oggetti della famiglia corretta.

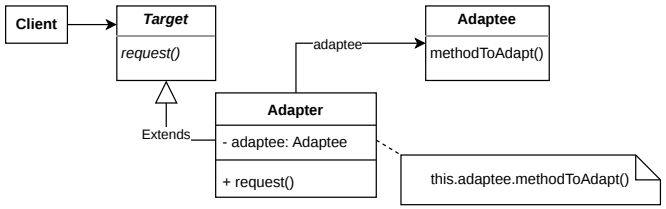
Pattern strutturali

I pattern strutturali sono quelli che si occupano di come le classi e gli oggetti vengono composti per formare strutture più grandi.

Adapter

L'**Adapter** è un pattern che permette di adattare un'interfaccia ad un'altra. Supponiamo di voler implementare una feature e, per farlo, ci serviamo di una libreria esterna. La libreria, però, lavora su dati e oggetti diversi da quelli con cui lavoriamo noi. In questo caso, possiamo creare un **Adapter** che si occupa di *adattare* l'interfaccia della libreria esterna all'interfaccia che usiamo.

UML



In questo diagramma, abbiamo `Target` che definisce l'interfaccia che il Client usa, `Adaptee` che definisce l'interfaccia che vogliamo *adattare* (ad esempio una libreria esterna), e `Adapter`, che estende `Target`, e usa `Adaptee` per adattare il methodo `methodToAdapt()` al metodo `request()` di `Target`.

Esempio

Supponiamo di voler implementare un programma che legge un file di testo e lo stampa a video. Per farlo, usiamo una libreria esterna, che però lavora su oggetti diversi da quelli che usiamo noi. In questo caso, possiamo creare un **Adapter** che si occupa di *adattare* l'interfaccia della libreria esterna all'interfaccia che usiamo.

Per prima cosa, definiamo l'interfaccia `Target`, in questo esempio la chiamiamo `TextPrinter`, che lavora con il tipo di file `TextFile`:

```
public class TextFile {
  private String name;
  private String path;
  private int size;

  public TextFile
  (String name, String path, int size) {
    this.name = name;
    this.path = path;
    this.size = size;
  }

  public String getPath() {
    return path;
  }

  ...
}

public interface TextPrinter {
  public void printText(TextFile textFile);
}
```

Definiamo anche l'interfaccia `Adaptee` (la libreria esterna), in questo esempio la chiamiamo `FilePrinterLibrary`, che userà un tipo di file diverso da `TextFile` (`File`):

```
public class File {
  private String path;

  public File(String path) {
    this.path = path;
  }

  public String getPath() {
    return path;
  }

  ...
}

public class FilePrinterLibrary {
  public void printFile(File file) {
    System.out.println(
      "Printing file " + file.getPath());
  }
}
```

`TextPrinter` e `FilePrinterLibrary` lavorano con oggetti diversi, quindi, per adattare l'interfaccia di `FilePrinterLibrary` a quella di `TextPrinter`, creiamo un **Adapter**:

```
public class FilePrinterLibraryAdapter
implements TextPrinter {
  FilePrinterLibrary filePrinterLibrary =
    new FilePrinterLibrary();

  @Override
  public void printText(TextFile textFile) {
    File file=new File(textFile.getPath());

    filePrinterLibrary.printFile(file);
  }
}
```

`FilePrinterLibraryAdapter` definisce il metodo `printText()` per poter implementare l'interfaccia `TextPrinter`. Implementiamo il metodo `printText()` utilizzando un oggetto `FilePrinterLibrary` e creando un oggetto `File` a partire da un oggetto `TextFile`, quindi possiamo chiamare il metodo `printFile()` di `FilePrinterLibrary`.

```
public class Main {
  public static void main(String[] args) {
    TextFile textFile = new TextFile(
      "test.txt", "/home/user", 100
    );

    TextPrinter textPrinter =
      new FilePrinterLibraryAdapter();

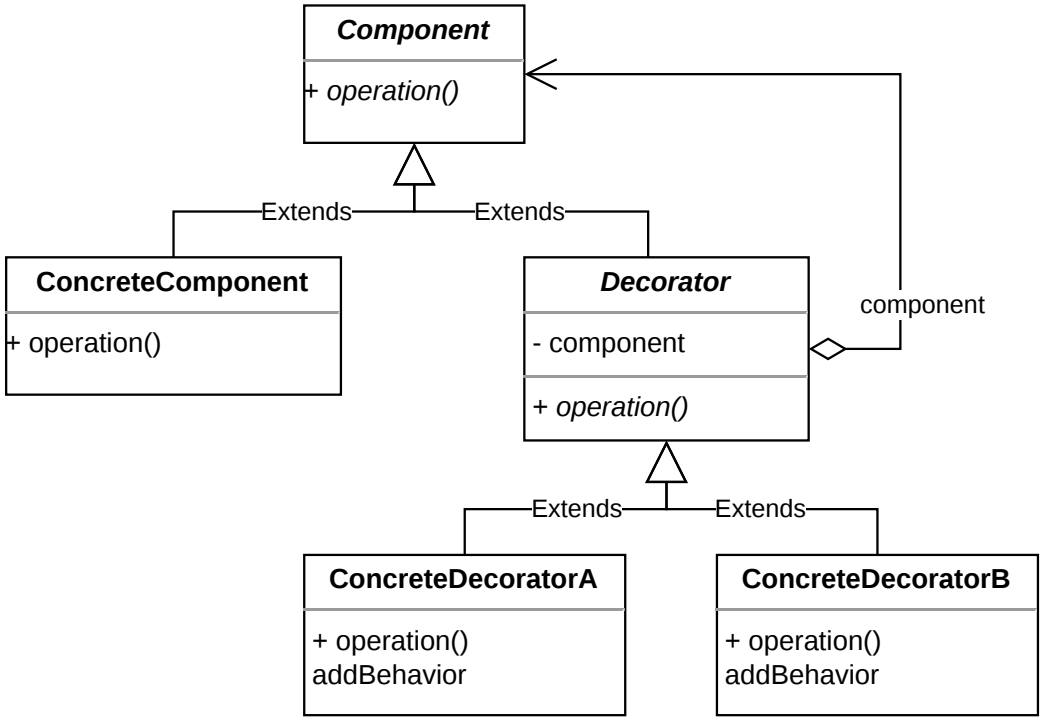
    textPrinter.printText(textFile);
  }
}
```

In questo modo, il `Main` usa il tipo di file `TextFile`, sarà poi l'**Adapter** che si occuperà di convertirlo in `File` per richiamare la funzione di libreria.

Decorator

Il **Decorator** è un pattern che permette di aggiungere dinamicamente una o più funzionalità ad un sistema.

UML



In questo diagramma, abbiamo `Component` che definisce l'interfaccia che il Client usa, `ConcreteComponent` che implementa `Component`, e `Decorator` che estende `Component` e usa `Component` per aggiungervi funzionalità, vi sono poi i vari `ConcreteDecorator` che estendono `Decorator` e realizzano (ciascuno) una funzionalità.

Si noti che l'attributo `component` di `Decorator` è di tipo `Component`, e non deve essere per forza un `ConcreteComponent`, ma anche un altro `Decorator` (`ConcreteDecoratorA` o `ConcreteDecoratorB`), che a sua volta avrà di nuovo un `component`. Si può quindi aggiungere quante funzionalità si vogliono al `ConcreteComponent` *base*.

Esempio

Supponiamo di voler costruire un programma che gestisce un'interfaccia per la visualizzazione di un testo. Il programma deve essere in grado di visualizzare il testo in modo semplice, in grassetto, ma anche cambiando il colore del testo, o tutti e due insieme. Per farlo, usiamo un **Decorator**.

Per prima cosa, definiamo l'interfaccia `Component`, in questo esempio la chiamiamo `TextView`, che definisce il metodo `draw()`:

```
public interface TextView {
  public void draw();
}
```

Definiamo anche la classe `ConcreteComponent`, in questo esempio la chiamiamo `SimpleTextView`, che implementa `TextView` (e stamperà semplicemente il testo che gli viene passato):

```
public class SimpleTextView implements TextView {
  private String text;

  public SimpleTextView(String text) {
    this.text = text;
  }

  @Override
  public void draw() {
    System.out.println(text);
  }
}
```

Definiamo la classe astratta `Decorator`, in questo esempio la chiamiamo `TextViewDecorator`, che estende `TextView` e usa `TextView` per aggiungere funzionalità:

```
public abstract class TextViewDecorator implements TextView {
  private TextView textView;

  public TextViewDecorator(TextView textView) {
    this.textView = textView;
  }

  @Override
  public void draw() {
    textView.draw();
  }
}
```

L'attributo `textView` di `TextViewDecorator` è di tipo `TextView`, e (come detto prima) può essere sia un `SimpleTextView` che un altro `ConcreteDecorator`.

Definiamo le classi `ConcreteDecoratorA`, in questo esempio la chiamiamo `ColoredTextView`; e `ConcreteDecoratorB`, in questo esempio la chiamiamo `BoldTextView`, che estendono `TextViewDecorator`:

```
public class ColoredTextView extends TextViewDecorator {
  private String color;

  public ColoredTextView(TextView textView, String color) {
    super(textView);
    this.color = color;
  }

  // Stampa il testo colorato
  @Override
  public void draw() {
    System.out.print(color);
    super.draw();
    System.out.print("\u001B[0m");
  }
}
```

```
public class BoldTextView extends TextViewDecorator {
  public BoldTextView(TextView textView) {
    super(textView);
  }

  // Stampa il testo in grassetto
  @Override
  public void draw() {
    System.out.print("\u001B[1m");
    super.draw();
    System.out.print("\u001B[0m");
  }
}
```

Si noti che `ColoredTextView` e `BoldTextView` hanno nel loro costruttore l'istruzione `super(textView)`, che chiama il costruttore di `TextViewDecorator` per settare l'attributo `textView` (che può essere sia un `SimpleTextView` che un `ColoredTextView` o un `BoldTextView`).

```
public class Main {
  public static void main(String[] args) {
    // Stampa "Hello World!" normale
    TextView textView = new SimpleTextView("Hello World!");
    textView.draw();

    // Stampa "Hello World!" in rosso
    TextView coloredTextView = new ColoredTextView(
      new SimpleTextView("Hello World!"), "\u001B[31m");
    coloredTextView.draw();

    // Stampa "Hello World!" in grassetto
    TextView boldTextView = new BoldTextView(
      new SimpleTextView("Hello World!"));
    boldTextView.draw();

    // Stampa "Hello World!" in rosso e in grassetto
    TextView coloredBoldTextView = new ColoredTextView(
      new BoldTextView(
        new SimpleTextView("Hello World!")
      ),
      "\u001B[31m"
    );
    coloredBoldTextView.draw();
  }
}
```

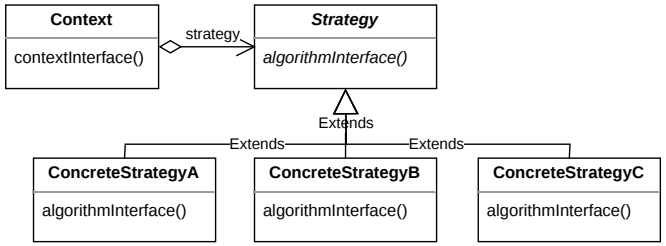
Con questa *catena* di **Decorator** è possibile aggiungere dinamicamente più funzionalità al `SimpleTextView` *base*.

Pattern comportamentali

Strategy

Strategy è un pattern che permette di cambiare dinamicamente il comportamento di un sistema.

UML



In questo diagramma, troviamo `Strategy` che definisce un comportamento che le varie `ConcreteStrategy` implementano, ognuno in modo diverso. `Context` usa una `Strategy` *generica*, che verrà assegnata dinamicamente dal client attraverso una `ConcreteStrategy`. Si noti che `Strategy` e `ConcreteStrategy` sono classi che hanno un solo metodo, sono quindi **interfacce funzionali**. Possiamo quindi usare le **espressioni lambda** per definire le `ConcreteStrategy` senza doverle creare esplicitamente.

Esempio (senza interfacce funzionali)

Supponiamo di voler gestire un sistema di apertura di casse in un gioco. Le casse possono dare, una volta aperte, degli effetti al giocatore che le apre; ad esempio, una cassa può curare il giocatore, un'altra può avvelenarlo, ecc. Usiamo uno **Strategy**. Per prima cosa, definiamo l'interfaccia `Strategy`, in questo esempio la chiamiamo `EffettoCassa`, che definisce il metodo `applicaEffetto()`:

```
public interface EffettoCassa {
  public void applicaEffetto();
}
```

Definiamo le varie classi `ConcreteStrategy`, in questo esempio le chiamiamo `EffettoCassaCurativa`, `EffettoCassaAvvelenata`, ecc., che implementano `EffettoCassa`:

```
public class EffettoCassaCurativa
implements EffettoCassa {
  @Override
  public void applicaEffetto() {
    System.out.println("Sei stato curato!");
  }
}
```

```
public class EffettoCassaAvvelenata
implements EffettoCassa {
  @Override
  public void applicaEffetto() {
    System.out.println("Sei avvelenato!");
  }
}
```

Definiamo la classe `Context`, in questo esempio la chiamiamo `Cassa`, che usa un `EffettoCassa` *generico* e chiama il metodo `apri()`:

```
public class Cassa {
  private EffettoCassa effettoCassa;

  public Cassa(EffettoCassa effettoCassa) {
    this.effettoCassa = effettoCassa;
  }

  public void apri() {
    effettoCassa.applicaEffetto();
  }
}
```

Il Main poi assegnerà una `ConcreteStrategy` (`EffettoCassaCurativa`, `EffettoCassaAvvelenata`, ecc.) a quella *generica* (`EffettoCassa`) nel costruttore di `Cassa`.

Esempio (con interfacce funzionali)

Usando le interfacce funzionali, possiamo usare una sola classe `Cassa`:

```
public class Cassa {
  Runnable effettoCassa;

  public Cassa(Runnable effettoCassa) {
    this.effettoCassa = effettoCassa;
  }

  public void apri() {
    effettoCassa.run();
  }
}
```

Per il nostro uso, definiamo una `Runnable`, che è un'interfaccia funzionale che ha un solo metodo: `run()` per eseguire il codice. A seconda dei tipi dei parametri e del tipo del valore restituito, possiamo usare anche le altre interfacce funzionali spiegate nel capitolo "Programmazione funzionale". Il `Main` può quindi definire le `Runnable` *concrete* tramite espressioni lambda e, richiamando il costruttore di `Cassa`, assegnarle a quella *generica* per eseguire il giusto comportamento:

```
public class Main {
  public static void main(String[] args) {
    // Crea una cassa curativa
    Cassa cassaCurativa =
    new Cassa(() -> {
      System.out.println("Sei stato curato!");
    });
    /* equivalente a:
    Runnable effettoCassaCurativa = () -> {
      System.out.println("Sei stato curato!");
    };
    Cassa cassaCurativa =
      new Cassa(effettoCassaCurativa);
    */

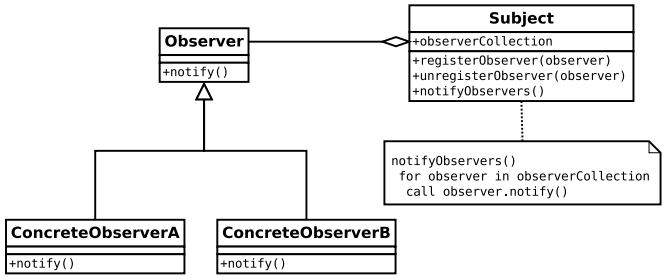
    // Crea una cassa avvelenata
    Cassa cassaAvvelenata =
    new Cassa(() -> {
      System.out.println("Sei avvelenato!");
    });

    // Apri le casse curativa e avvelenata
    cassaCurativa.apri();
    cassaAvvelenata.apri();
  }
}
```

Observer

Observer è un pattern che permette di notificare ad un oggetto quando un altro oggetto cambia.

UML



In questo diagramma, troviamo `Subject` che è l'oggetto che cambia e notifica gli `Observer`; gli `Observer` che ricevono la notifica e si comportano conseguentemente. Infine abbiamo i `ConcreteObserver` che implementano `Observer` e definiscono il comportamento che si attiva quando `Subject` cambia. Inoltre `Subject` può essere astratta ed avere una `ConcreteSubject` che la estende.

Esempio

Supponiamo di voler costruire un sistema di gestione di dati di un'azienda. L'azienda ha un database che contiene i dati dei dipendenti, e un sistema che permette di visualizzare questi dati. Il sistema di visualizzazione deve essere aggiornato ogni volta che il database cambia. Usiamo un **Observer**.

Per prima cosa, definiamo l'interfaccia `Observer`, in questo esempio la chiamiamo `Visualizzazione`, che definisce il metodo `aggiorna()`:

```
public interface Visualizzazione {
  public void aggiorna();
}
```

Definiamo le varie classi `ConcreteObserver`, in questo esempio le chiamiamo `VisualizzazioneConsole`, `VisualizzazioneGUI`, ecc., che implementano `Visualizzazione`:

```
public class VisualizzazioneConsole
implements Visualizzazione {
  @Override
  public void aggiorna() {
    System.out.println(
      "Aggiornamento della
      visualizzazione console");
  }
}
```

```
public class VisualizzazioneGUI
implements Visualizzazione {
  @Override
  public void aggiorna() {
    System.out.println(
      "Aggiornamento della
      visualizzazione GUI");
  }
}
```

Definiamo la classe `Subject`, in questo esempio la chiamiamo `Database`, che ha un `ArrayList` di `Visualizzazione` e un metodo `aggiornaVisualizzazioni()` che chiama il metodo `aggiorna()` di ogni `Visualizzazione`:

```
public class Database {
  private ArrayList<Visualizzazione>
    visualizzazioni;

  public Database() {
    visualizzazioni = new ArrayList<>();
  }

  public void aggiungiVisualizzazione
  (Visualizzazione visualizzazione) {
    visualizzazioni.add(visualizzazione);
  }

  public void rimuoviVisualizzazione
  (Visualizzazione visualizzazione) {
    visualizzazioni.remove(visualizzazione);
  }

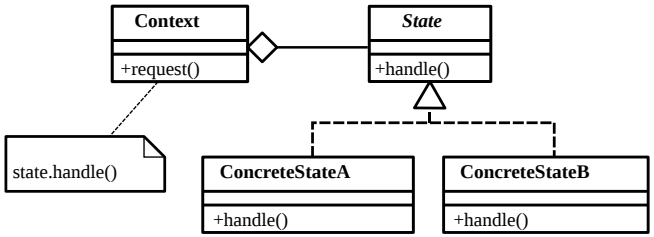
  protected void aggiornaVisualizzazioni() {
    for(Visualizzazione visualizzazione
      : visualizzazioni) {
      visualizzazione.aggiorna();
    }
  }
}
```

Si noti che nei metodi per aggiungere o rimuovere una `Visualizzazione`, abbiamo l'interfaccia `Visualizzazione` come tipo di parametro. Questo ci permette di aggiungere dinamicamente qualsiasi tipo di `Visualizzazione` che implementa l'interfaccia `Visualizzazione`.

State

State è un pattern che permette di cambiare il comportamento di un oggetto in base allo stato in cui si trova. È molto simile a **Strategy**.

UML



In questo diagramma, troviamo `State` che è l'interfaccia che definisce uno stato *generico*, `ConcreteState` che estende `State` e definisce il comportamento in quello specifico stato e `Context` che è l'oggetto che cambia stato.

Esempio

Supponiamo di voler implementare il TCP. La sequenza di azioni da compiere è rappresentabile con un automa a stati finiti. La classe TCPConnection deve gestire l'apertura della connessione, la chiusura e le varie fasi intermedie. Usiamo uno **State**.

Definiamo la classe astratta `State`, in questo esempio la chiamiamo `TCPState`, che definisce i metodi `open()`, `close()`, `acknowledge()` e `transition()`:

```
public abstract class TCPState {
  protected TCPConnection connection;

  public TCPState(TCPConnection connection) {
    this.connection = connection;
  }

  public abstract void open();
  public abstract void close();
  public abstract void acknowledge();

  public void transition(TCPState state) {
    connection.setState(state);
  }
}
```

Definiamo le varie classi `ConcreteState`, in questo esempio le chiamiamo `TCPListen`, ecc., che estendono `TCPState`:

```
public class TCPListen extends TCPState {
  public TCPListen(TCPConnection connection) {
    super(connection);
  }

  @Override
  public void open() {
    // open connection
    this.transition(
      new TCPEstablished(connection));
  }
  @Override
  public void close() { ... }
  @Override
  public void acknowledge() { ... }
}
```

Osserviamo la differenza **fondamentale** tra *Strategy* e *State*. Alla fine dell'esecuzione del metodo open, avviene una transizione dello stato da `TCPListen` a `TCPEstablished`. Infatti sia in *Strategy* che in *State* il comportamento della classe cambia a runtime, la differenza tra i due è che nel primo caso è l'utente che decide quando cambiare il comportamento, nel secondo caso il comportamento cambia perchè avviene una transizione di stato che è completamente trasparente all'utente della classe e dipende solo dallo stato di partenza e dal metodo invocato.

Definiamo la classe `Context`, in questo esempio la chiamiamo `TCPConnection`, che ha un campo `state` di tipo `TCPState` e i metodi `open()`, `close()`, `acknowledge()` e `setState()`:

```
public class TCPConnection {
  private TCPState state;

  public TCPConnection() {
    state = new TCPListen(this);
  }

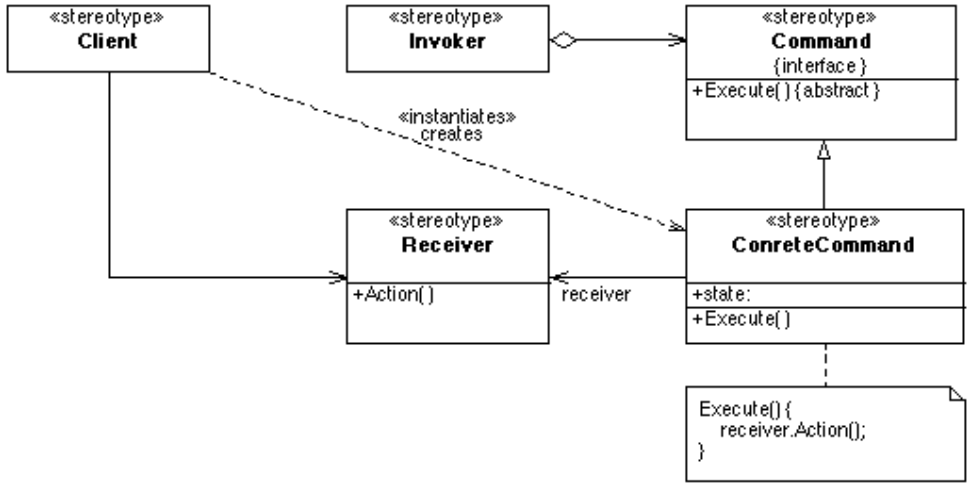
  public void open() {
    state.open();
  }
  public void close() {
    state.close();
  }
  public void acknowledge() {
    state.acknowledge();
  }

  public void setState(TCPState newState) {
    this.state = newState;
  }
}
```

Command

Il pattern **Command** permette di isolare la porzione di codice che effettua un'azione (eventualmente molto complessa) dal codice che ne richiede l'esecuzione; l'azione è incapsulata nell'oggetto Command. L'obiettivo è rendere variabile l'azione del client senza però conoscere i dettagli dell'operazione stessa. Altro aspetto importante è che il destinatario della richiesta può non essere deciso staticamente all'atto dell'istanziazione del command ma ricavato a runtime.

UML



La classe `Invoker` non implementa la richiesta direttamente ma la delega ad un oggetto `Command` che la esegue. Il `ConcreteCommand` implementa `Command` e performa l'azione sul `Receiver`.

Esempio

Supponiamo di creare una classe Menu per eseguire operazioni all'interno di una applicazione. Un'istanza di Menu contiene un insieme di MenuItem, uno per ciascuna azione da eseguire. Una istanza di Menu non deve conoscere i dettagli dell'applicazione associata, né delle azioni associate ai MenuItem. Usiamo il pattern **Command**.

Definiamo l'interfaccia `Command` che definisce il metodo `esegui()`:

```
interface Command {
  void esegui();
}
```

Definiamo la classe `PasteCommand` che implementa `Command`:

```
class PasteCommand implements Command {
  private Document doc;

  public PasteCommand(Document doc) {
    this.doc = doc;
  }

  @Override
  public void esegui() {
    ...
  }
}
```

Definiamo la classe `Menu` che contiene un insieme di `MenuItem`:

```
class Menu {
  List<MenuItem> items = new ArrayList<>();

  public void addMenuItem(MenuItem item) {
    ...
  }
}
```

Definiamo la classe `MenuItem` che contiene un `Command`:

```
class MenuItem {
  Command command;

  public void setCommand(Command command) {
    this.command = command;
  }

  public void click() {
    command.esegui();
  }
}
```

Si noti che l'azione da compiere è slegata dagli oggetti che la invocano.

Testing

Testing strutturale (white-box)

Il **testing strutturale** tiene conto della struttura interna del programma con l'obiettivo di sollecitarne tutte le parti.

Criterio di copertura delle istruzioni (statement coverage)

Si vuole fare in modo che ogni istruzione del programma venga eseguita almeno una volta. Consideriamo la seguente funzione:

```
int sqrt(int n) {
  int i = 1, z = 1;
  if(n < 0)
    return -1;
  while(z <= n) {
    i = i + 1;
    z = i * i;
  }
  return i - 1;
}
```

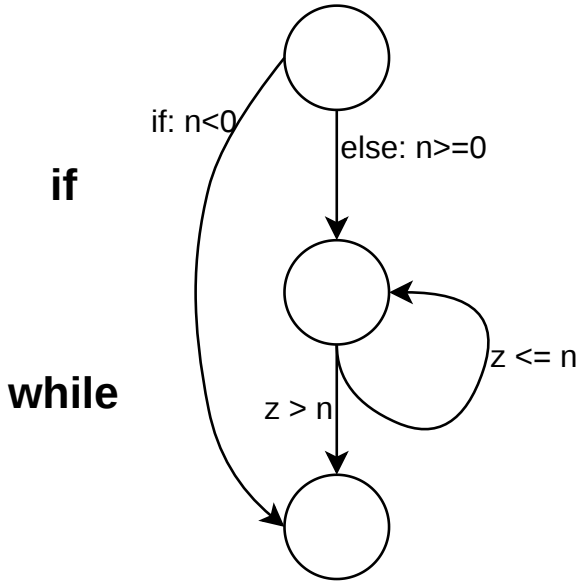
Dobbiamo selezionare dei casi di test in modo che ogni istruzione venga eseguita almeno una volta. possiamo scegliere i seguenti casi di test:

- `n = -1` per eseguire il corpo dell'`if`
- `n = 1` per eseguire il corpo del `while`

Osservazione: se una funzione ha n `return` allora avremo per forza **almeno n casi di test**.

Criterio di copertura delle decisioni (edge coverage)

Si vuole coprire tutte le possibili *diramazioni* del programma. Per fare ciò è utile costruire il **control flow graph** (o **CFG**) del programma in cui si evidenziano proprio tutte le diramazioni. Nella figura seguente è riportato il CFG della funzione `sqrt` (esempio precedente):



Ogni arco rappresenta una possibile diramazione. Si noti che ogni `if` e `while` è rappresentato da due archi, uno per il caso `true` e uno per il caso `false` (anche se ad esempio il ramo `else` non è esplicitamente presente nel codice).

Il criterio **edge coverage** richiede che ogni arco del CFG venga eseguito almeno una volta. Per fare ciò possiamo scegliere i seguenti casi di test:

- `n = -1` per eseguire solo l'arco `n < 0` (e giungere alla terminazione del programma)
- `n = 1` per eseguire il ramo else del primo `if` e per entrare nel `while` (e quindi eseguire tutti e due gli archi del `while`)

Criterio di copertura delle decisioni e delle condizioni (edge and condition coverage)

L'insieme dei casi di test deve essere definito in modo che ogni ramo del CFG venga attraversato almeno una volta e sollecitando tutti i valori di verità delle sottoespressioni che compaiono nelle condizioni composte, tenendo conto della *short circuit evaluation*.

Per esempio, consideriamo la seguente funzione:

```
void p(int x, int y) {
  if(x == 0 || y > 0)
    y = y/x;
  else
    y = (-y)/x;
}
```

Si può entrare nel ramo `if` nei seguenti modi: `x = 0, y qualsiasi` e `x != 0, y > 0`

e nel ramo `else`: `x != 0` e `y <= 0`.

Sono quindi sufficienti 3 casi di test: `x = 0, y = -1`, `x = 1, y = 1` e `x = 1, y = -1`.

Si noti che la short circuit evaluation implica che servono tre casi di test anche con il seguente programma:

```
void p(int x, int y) {
  if(x == 0 && y > 0)
    y = y/x;
  else
    y = (-y)/x;
}
```

Si può entrare nel ramo `if` solamente se: `x = 0, y > 0`

e nel ramo `else` con: `x != 0, y qualsiasi` e `x = 0, y <= 0`.

Servono quindi anche in questo caso 3 casi di test: `x = 0, y = 1`, `x = 1, y = 1` e `x = 0, y = -1`.

Criterio di copertura dei cammini (path coverage)

L'insieme dei casi di test deve garantire che ogni possibile cammino (o percorso) che porti dal nodo iniziale al nodo finale del CFG sia attraversato almeno una volta.

Attenzione: il numero di volte in cui si esegue un ciclo (un autoanello nel CFG) è rilevante. Perciò, se un ciclo può essere eseguito "infinite" volte, allora il criterio di copertura dei cammini non è applicabile. Ad esempio considerando la funzione di prima `int sqrt(int n)`, `n` può essere grande "quanto vogliamo" (in realtà non è così, dato che è un intero a 32 bit, ma supponiamo che sia così), pertanto il ciclo `while` può essere eseguito "infinite volte". Il criterio non è quindi applicabile.

Consideriamo, per esempio, la seguente funzione:

```
int f(int x, int y) {
  int z = 0;
  if(x > 0) {
    if(y > 0) {
      z = x + y;
    } else {
      z = x - y;
    }
  } else {
    if(y > 0) {
      z = x * y;
    } else {
      z = x / y;
    }
  }
  return z;
}
```

I possibili cammini sono 4:

- `x > 0, y > 0` prendendo il primo ramo dell'`if` esterno e il primo ramo dell'`if` interno al ramo `if`
- `x > 0, y <= 0` prendendo il primo ramo dell'`if` esterno e il secondo ramo `else` interno al ramo `if`
- `x <= 0, y > 0` prendendo il secondo ramo `else` esterno e il primo ramo dell'`if` interno al ramo `else`
- `x <= 0, y <= 0` prendendo il secondo ramo `else` esterno e il secondo ramo `else` interno al ramo `else`

Esercizio sul testing (TdE del 2020-01-16, esercizio 4)

Si consideri il seguente frammento di codice:

```
int f(int a, int b) {
  if(a > b) {
    System.out.println("A");
  } else {
    System.out.println("B");
  }
  for(int i = 0; i < 1000; i++) {
    if(a > 0) break;
    else a = 1;
  }
}
```

Domanda a)

Definire un insieme minimo di casi di test per coprire tutte le istruzioni (statement coverage)

Soluzione

Sono sufficienti 2 casi di test:

- `a = 1, b = 0` per eseguire il ramo `if` del primo `if` e il `break` interno al `for`
- `a = 0, b = 1` per eseguire il ramo `else` del primo `if` e sia `a=1` che il `break` nel `for`

Domanda b)

Definire un insieme minimo di casi di test per coprire tutte le decisioni (edge coverage)

Soluzione

Si noti che il ciclo `for` può essere eseguito al massimo 2 volte e poi il `break` lo interrompe. Non è quindi possibile coprire la decisione `i<1000`. Tutte le altre decisioni sono coperte dai 2 casi precedenti.

Domanda c)

Definire un insieme minimo di casi di test per coprire tutti i cammini (path coverage)

Soluzione

Quindi, il corpo del `for` può essere eseguito 1 volta (se si entra direttamente nell'`if` e si esegue il break) o 2 volte (se si entra nell'`else`, per poi eseguire `break` la seconda iterazione). Per cui abbiamo 4 possibili cammini (quindi 4 casi di test):

- `a = 1, b = 0` per eseguire il primo `if` e il corpo del `for` una volta
- `a = 1, b = 2` per eseguire il primo `else` e il corpo del `for` una volta
- `a = -1, b = 0` per eseguire il primo `else` e il corpo del `for` due volte
- `a = -1, b = -2` per eseguire il primo `if` e il corpo del `for` due volte

UML